

ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN

ĐỒ ÁN MÔN HỌC

Môn: Xử lý ảnh và ứng dụng
Đề tài:

XÂY DỰNG HỆ THỐNG NHẬN DIỆN BIỂN SỐ XE BẰNG TRÍ TUỆ NHÂN TẠO

Sinh viên thực hiện:	Phạm Nguyễn Thế Châu — MSSV 25410004
	Nguyễn Công Hậu — MSSV 25410006
	Nguyễn Minh Hiếu — MSSV 25410007
	Phạm Công Thành — MSSV 25410013
Lớp:	AI503.F3.LT.TTNT
Giảng viên hướng dẫn:	ThS. Cáp Phạm Đình Thắng

TP. Hồ Chí Minh, tháng 9 năm 2026

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU	4
1.1. Đặt vấn đề	4
1.2. Mục tiêu và phạm vi	4
1.3. Lựa chọn công nghệ	5
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT	7
2.1. Quy chuẩn biển số xe Việt Nam và hệ quả hình học	7
2.2. Các phép xử lý ảnh sử dụng trong đồ án	9
2.3. Phát hiện vùng biển số	11
2.4. Nhận dạng ký tự và điểm suy giảm trên văn bản hai dòng	12
CHƯƠNG 3. THIẾT KẾ VÀ CÀI ĐẶT	14
3.1. Kiến trúc tổng thể	14
3.2. Xây dựng bộ dữ liệu	14
3.3. Huấn luyện bộ phát hiện	16
3.4. Khối xử lý ảnh vùng biển số	17
3.5. Phân loại màu nền trong không gian HSV	21
3.6. Bộ luật hậu xử lý theo vị trí	22
3.7. Ứng dụng trình diễn	23
CHƯƠNG 4. THỰC NGHIỆM VÀ ĐÁNH GIÁ	26
4.1. Môi trường, dữ liệu và quy ước	26
4.2. Kết quả phát hiện vùng biển	26
4.3. Kết quả nhận dạng ký tự	28
4.4. Bóc tách đóng góp của từng bước xử lý ảnh	31
4.5. Hiệu năng	33
4.6. Phân tích lỗi	34
4.7. Các yếu tố ảnh hưởng tới tính hợp lệ của kết quả	35
CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	37
5.1. Kết quả đạt được	37
5.2. Hạn chế	37
5.3. Hướng phát triển	38
5.4. Kết luận chung	38
TÀI LIỆU THAM KHẢO	40

PHỤ LỤC	42
Phụ lục A. Cấu hình và siêu tham số.....	42
Phụ lục B. Hướng dẫn cài đặt và chạy	43
Phụ lục C. Phân công công việc	44

CHƯƠNG 1. GIỚI THIỆU

1.1. Đặt vấn đề

Tính đến tháng 9/2024, Việt Nam có khoảng **77 triệu xe máy đã đăng ký**, chiếm **85–90% lưu lượng phương tiện trên đường** [1]. Con số này là một ràng buộc kỹ thuật trực tiếp chứ không phải bối cảnh xã hội: mọi xe mô tô đều mang **biển hai dòng gần vuông**, nên ở Việt Nam biển hai dòng là dạng phổ biến chứ không phải ngoại lệ. Biển xe mô tô chỉ 190×140 mm, nên đây đồng thời là bài toán phát hiện **đối tượng nhỏ**.

Nhận dạng biển số xe tự động (ALPR) là lõi của bãi đỗ xe thông minh, thu phí không dừng, kiểm soát ra vào và giám sát giao thông. Cả bốn ứng dụng đều đo cùng một đại lượng: **tỉ lệ đọc đúng toàn bộ chuỗi biển số**, chứ không phải tỉ lệ đọc đúng từng ký tự — đọc đúng 9 trên 10 ký tự vẫn là đọc sai biển.

Không thể dùng trực tiếp giải pháp nước ngoài, vì ba lý do: một đo được, một pháp lý, một thuộc điều kiện thu nhận ảnh.

Thứ nhất, biển hai dòng là điểm suy giảm đã đo được. Trên tập kiểm thử cân bằng của bộ **RodoSol-ALPR (Brazil)** — 4.000 ảnh ô tô biển một dòng, 4.000 ảnh xe máy biển hai dòng — hệ thống thương mại OpenALPR nhận đúng **94,3% biển một dòng nhưng chỉ 45,7% biển hai dòng**, chênh **48,6 điểm phần trăm**, không biển số nào khác thay đổi ngoài bố cục biển [2]. Cặp số này đo trên **dữ liệu Brazil, không phải dữ liệu Việt Nam**; đồ án dẫn nó như một dẫn chứng định lượng về độ khó của biển hai dòng tại một quốc gia cũng có tỉ lệ xe máy cao, không phải như mốc chuẩn.

Thứ hai, cấu trúc chuỗi và hình học biển là đặc thù quốc gia. Biển số Việt Nam theo Thông tư 79/2024/TT-BCA [3], mã tỉnh theo Thông tư 51/2025/TT-BCA [4], kích thước vật lý theo QCVN 08:2024/BCA [5]. Ba đặc thù ở Chương 2 không học được từ dữ liệu nước ngoài, trong đó **tỉ lệ khung hình** là đại lượng thuần hình học mà toàn bộ khối xử lý ảnh của đồ án dựa vào.

Thứ ba, điều kiện thu nhận ảnh khác biệt: biển bám bụi, cong vênh, chụp nghiêng, ngược sáng, ảnh đêm, bề mặt phản quang gây chói cục bộ — đúng nhóm vấn đề mà môn Xử lý ảnh cung cấp công cụ để giải.

1.2. Mục tiêu và phạm vi

1.2.1. Mục tiêu

Xây dựng một hệ thống nhận dạng biển số xe Việt Nam chạy được đầu cuối, **suy luận hoàn toàn trên CPU**, hỗ trợ cả biển một dòng và biển hai dòng. Trọng tâm của đồ án môn học đặt ở **khối xử lý ảnh** nằm giữa bộ phát hiện và bộ nhận dạng ký tự: chuẩn hoá, tăng cường tương phản, khử nhiễu bảo toàn biên, nắn hình, phân loại bố cục theo hình học, tách và ghép ảnh, phân tích màu trong không gian HSV (Bảng 1.1).

Bảng 1.1. Chỉ tiêu đặt ra, mỗi chỉ tiêu có ngưỡng tối thiểu và mục tiêu

Chỉ tiêu đánh giá	Ngưỡng tối thiểu	Mục tiêu
mAP@0,5 của bộ phát hiện biển số	0,85	0,90
mAP@0,5:0,95 của bộ phát hiện	0,55	0,65
Độ chính xác mức ký tự (1 – CER)	0,92	0,95
Đúng cả chuỗi, trước hậu xử lý	0,80	0,85
Đúng cả chuỗi, sau hậu xử lý	0,85	0,90
Độ trễ xử lý một ảnh, p95, trên CPU	≤ 1.500 ms	≤ 800 ms

Hai chỉ tiêu *trước* và *sau* hậu xử lý được đo tách bạch có chủ đích: **hiệu số giữa chúng chính là đóng góp định lượng của khối hậu xử lý**, đại lượng mà phần lớn tài liệu chỉ mô tả định tính.

Ngưỡng độ trễ rộng hơn các bài báo ALPR vì máy thực hiện **không có GPU CUDA**: huấn luyện chạy trên GPU đám mây, còn toàn bộ suy luận và trình diễn chạy trên CPU, trong khi các bài báo thường đo trên GPU rời. Mọi số liệu hiệu năng trong đề án vì vậy bắt buộc kèm cấu hình phần cứng.

1.2.2. Phạm vi

Trong phạm vi: thu thập và làm sạch bộ dữ liệu ảnh; khử trùng lặp bằng băm tri giác; huấn luyện bộ phát hiện; toàn bộ khối xử lý ảnh vùng biển; bộ luật hậu xử lý chuỗi theo quy chuẩn Việt Nam; phân loại màu nền; đánh giá đầy đủ kèm phân tích lỗi; một ứng dụng web tối thiểu để trình diễn.

Ngoài phạm vi: xác thực và phân quyền (hệ thống chạy nội bộ); bám vết đối tượng qua khung hình; ước lượng tốc độ và phát hiện vi phạm; biển số nước ngoài; huấn luyện lại bộ nhận dạng ký tự — cấu hình triển khai dùng nguyên mô hình tiền huấn luyện; một lượt tinh chỉnh đã thử và bị loại vì kém hơn bản gốc ở cấu hình vận hành; suy luận trên GPU.

1.3. Lựa chọn công nghệ

Mỗi lựa chọn dưới đây bị chi phối bởi cùng bốn ràng buộc: **không có GPU, phải xử lý được biển hai dòng, phải đóng gói hoàn chỉnh có thể triển khai**, và **ngân sách thời gian máy hữu hạn** (Bảng 1.2).

Bảng 1.2. Các quyết định công nghệ và lý do

Hạng mục	Chọn	Phương án đã xét	Lý do chính	Đánh đổi
Thư viện xử lý ảnh	OpenCV [16]	scikit-image, Pillow	Đủ cả CLAHE, lọc song phương, biến đổi phối cảnh, HSV trong một thư viện; ràng buộc thời gian thực	API kiểu C cũ, dễ nhầm thứ tự kênh BGR/RGB

Hạng mục	Chọn	Phương án đã xét	Lý do chính	Đánh đổi
Bộ phát hiện	YOLO11n [7]	Faster R-CNN, SSD, YOLOv8	Họ một giai đoạn, anchor-free — hồi quy trực tiếp khoảng cách tâm tới bốn cạnh nên xử lý được cả tỉ lệ 4,7:1 lẫn 1,4:1 bằng một cơ chế; biến thể n chỉ 2,59 triệu tham số	Họ hai giai đoạn chính xác hơn nhưng không hợp ràng buộc CPU
Nhận dạng ký tự	PaddleOCR PP-OCRv5 mobile [8]	EasyOCR, Tesseract	Cao hơn hẳn hai bộ nhận dạng kia trên chính ảnh biển số Việt Nam, trong cấu hình đánh giá của đề án (mục 4.3.4)	Là đường ống nhiều giai đoạn thiết kế cho ảnh tài liệu, nên trả chi phí cho năng lực mà vùng biển đã cắt không cần
Hậu xử lý	Bộ luật tự thiết kế	Mô hình ngôn ngữ, từ điển	Biển số không có từ vựng để dựa vào; ràng buộc cú pháp lại rất chặt và kiểm được bằng biểu thức chính quy	Phải cập nhật khi văn bản pháp quy thay đổi
Ứng dụng trình diễn	FastAPI + React + Docker	Notebook, ứng dụng desktop	Yêu cầu chạy được bằng một lệnh trên máy sạch	Không phải trọng tâm của môn học

Việc chọn **PaddleOCR** là kết quả của một phép đo do đề án tự chạy chứ không phải suy đoán từ tài liệu: một số tài liệu công khai nghiêng về EasyOCR, còn phép đo trên 2.801 biển số Việt Nam ở mục 4.3.4 cho kết quả ngược lại **trong cấu hình đánh giá của đề án**.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

Chương này chỉ trình bày phần lý thuyết **ràng buộc trực tiếp một quyết định cài đặt ở** Chương 3. Thứ tự trình bày đi từ đối tượng (quy chuẩn biển số và hệ quả hình học của nó), qua các phép xử lý ảnh được sử dụng, rồi tới hai mô hình học sâu mà đề án dùng như công cụ.

2.1. Quy chuẩn biển số xe Việt Nam và hệ quả hình học

2.1.1. Cấu trúc chuỗi ký tự

Biển số ô tô trong nước gồm **8 ký tự**, ba thành phần: **mã địa phương** 2 chữ số, **seri** 1 chữ cái, **số thứ tự** 5 chữ số — ví dụ 30A-123.45 [3]. Trên đường vẫn còn biển 4 chữ số kiểu cũ (29A-1234) và xe đã đăng ký không bắt buộc đổi biển, nên biểu thức chính quy phải chấp nhận nhóm thứ tự **4 hoặc 5 chữ số**. Biển xe mô tô có **9 ký tự**, seri hai ký tự.

Mã địa phương hữu hạn và có lỗi hồng. Dải 11–99 có 89 giá trị, nhưng chỉ **81 mã đang được sử dụng**; tám mã **13, 42, 44, 45, 46, 87, 91, 96** không được gán [4]. Kiểm tra mã tỉnh vì vậy biển lỗi đọc ở hai vị trí đầu từ **sai âm thầm** thành **sai phát hiện được**: đọc ra 46A-123.45 thì biết ngay mã 46 không tồn tại.

Tập ký tự seri phụ thuộc vị trí. Đây là chi tiết dễ trình bày sai nhất. Biển trắng và vàng dùng seri thuộc tập **20 chữ cái** [6]; đối chiếu 26 chữ Latin thì vắng I J O Q R W. Nhưng suy diễn " $26 - 20 = 6$ chữ bị loại trừ" là **sai**: danh sách 20 chữ chỉ áp dụng cho **chữ cái thứ nhất**; ở **vị trí thứ hai** của seri xe máy là một tập khác — **có R, không có G**. Hợp hai vị trí, tập chữ không bao giờ xuất hiện trên biển Việt Nam chỉ gồm **5 chữ: I, J, O, Q, W** (Bảng 2.1).

Bảng 2.1. Tổng hợp các tập ký tự seri

Tập	Số lượng	Nội dung
Chữ cái thứ nhất của seri	20	A B C D E F G H K L M N P S T U V X Y Z
Chữ cái thứ hai của seri xe máy	20	A B C D E F H K L M N P R S T U V X Y Z
Seri biển nền xanh	11	A B C D E F G H K L M
Bị loại trừ khỏi toàn hệ thống	5	I, J, O, Q, W

Hệ quả cài đặt: **nếu huấn luyện lại bộ nhận dạng thì phải dùng đủ 36 ký tự** rồi mới ràng buộc ở tầng hậu xử lý. (*Mô hình đề xuất dùng nguyên mô hình PP-OCrv5, tập ký tự còn rộng hơn 36; ràng buộc hợp lệ vẫn đặt ở tầng hậu xử lý.*) Một mô hình huấn luyện trên tập ký tự 20 chữ cái sẽ **không bao giờ dự đoán được R**, gây sai sót có hệ thống trên mọi biển xe máy mang ký tự này — loại sai sót mà hậu xử lý không cứu được vì thông tin đã bị loại ngay ở tầng mô hình.

2.1.2. Kích thước vật lý và tỉ lệ khung hình

Đây là cơ sở hình học quan trọng nhất của đề án, vì nó cho phép **phân biệt biển một dòng với biển hai dòng bằng một đại lượng đo trực tiếp từ ảnh**, không cần huấn luyện thêm mô hình nào (Bảng 2.2).

Bảng 2.2. Kích thước và tỉ lệ khung hình các loại biển số [5]

Loại biển	Kích thước (dài × cao)	Tỉ lệ khung hình	Số dòng
Ô tô — biển dài	520 × 110 mm	4,727	1 dòng
Ô tô — biển ngắn	330 × 165 mm	2,000	2 dòng
Xe mô tô, xe gắn máy	190 × 140 mm	1,357	2 dòng

Ba giá trị này để lại một **khoảng trống rộng 2,727 đơn vị** giữa 2,000 và 4,727 mà không loại biển nào rơi vào. Khoảng trống đó là căn cứ để đặt ngưỡng phân loại bố cục ở mục 3.4.3.

Cần lưu ý mốc hiệu lực: bộ số liệu trên **chỉ đúng từ 01/01/2025**. Tiêu chuẩn trước đó quy định biển ô tô ngắn 200 × 280 mm và biển dài 110 × 470 mm, và nhiều tài liệu thư cấp vẫn dùng bộ số cũ.

Một hệ quả nữa: ô tô được cấp **hai** biển mang cùng chuỗi ký tự nhưng **hình dạng hoàn toàn khác nhau** (một dài một ngắn), trong khi xe mô tô chỉ có một biển hai dòng. Do đó số dòng bằng một *chứng minh* biển thuộc ô tô, còn số dòng bằng hai *không chứng minh gì* — cả ô tô lẫn xe máy đều có thể (Bảng 2.3).

2.1.3. Màu nền và giới hạn của thông tin ký tự

Bảng 2.3. Màu nền biển số và đối tượng áp dụng [6]

Màu nền / màu chữ	Đối tượng	Ghi chú
Trắng / đen	Cá nhân, tổ chức trong nước, xe không kinh doanh vận tải	Phổ biến nhất
Vàng / đen	Xe kinh doanh vận tải	Cùng cấu trúc ký tự với biển trắng
Xanh dương / trắng	Cơ quan Đảng, Nhà nước, công an	Seri chỉ dùng 11 chữ cái
Trắng / đỏ	Xe ngoại giao, tổ chức quốc tế	Cấu trúc chuỗi khác hẳn

Bảng này chỉ ra một giới hạn về nguyên tắc: biển vàng của xe kinh doanh vận tải mang **đúng cùng cấu trúc ký tự** với biển trắng của xe cá nhân, nên **không biểu thức chính quy nào phân biệt được**. Ngược lại, biển ngoại giao có nền trắng giống biển cá nhân nên riêng màu nền cũng không đủ. Chỉ **cặp thuộc tính gồm chuỗi ký tự và màu nền** mới định danh được loại phương tiện — lý do đề án xây dựng thêm một bộ phân loại màu trong không gian HSV (mục 3.5).

2.2. Các phép xử lý ảnh sử dụng trong đồ án

2.2.1. Chuyển thang xám

Ký tự trên biển số không mang thông tin phân biệt trong kênh màu: chữ đen trên nền trắng, đen trên nền vàng, trắng trên nền xanh — trong mọi trường hợp thông tin nằm ở **độ chói**, không ở sắc độ. Chuyển sang thang xám theo trọng số cảm thụ

$$Y = 0,299R + 0,587G + 0,114B$$

giảm ba kênh còn một, tức giảm hai phần ba khối lượng tính toán cho mọi bước phía sau mà không mất thông tin dùng được. Cần lưu ý: bước này chạy trên **nhánh nhận dạng ký tự**; nhánh phân loại màu nền (mục 3.5) làm việc trên ảnh màu gốc, vì đó chính là thông tin nó cần.

2.2.2. Cân bằng lược đồ xám thích nghi có giới hạn tương phản (CLAHE)

Bề mặt biển số Việt Nam **phản quang theo tiêu chuẩn**, nên dưới đèn pha hoặc nắng gắt nó tạo ra các **mảng chói cục bộ**: một phần biển bị bão hoà trắng trong khi phần còn lại vẫn tối. Cân bằng lược đồ xám toàn cục không xử lý được tình huống này, vì nó áp một hàm biến đổi duy nhất cho toàn ảnh — làm sáng vùng tối thì đồng thời đẩy vùng chói bão hoà thêm.

Cân bằng lược đồ xám thích nghi (AHE) chia ảnh thành các ô nhỏ và cân bằng riêng từng ô, nên vùng chói và vùng tối được xử lý bằng hai hàm biến đổi khác nhau. Nhược điểm của nó là **khuếch đại nhiễu** ở những ô gần đồng nhất: khi lược đồ tập trung vào vài mức xám, hàm phân phối tích lũy dừng đứng và một chênh lệch một mức xám bị kéo giãn thành chênh lệch lớn.

CLAHE [10] khắc phục bằng cách **cắt ngọn lược đồ** tại một hệ số giới hạn rồi **phân phối lại** phần bị cắt đều cho mọi mức xám, trước khi tính hàm phân phối tích lũy. Việc cắt ngọn đặt trần cho độ dốc của hàm biến đổi, tức đặt trần cho mức khuếch đại nhiễu.

Đồ án dùng hệ số giới hạn **2,0** trên lưới ô **8 × 8**. Kỹ thuật này đã được ghi nhận hiệu quả trong chính bài toán ALPR [9].

2.2.3. Lọc song phương thay cho làm mờ Gauss

Ảnh biển số do bộ phát hiện cắt ra thường nhoè và nhiễu, nên cần một bước khử nhiễu. Làm mờ Gauss

$$G(x, y) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right)$$

lấy trung bình có trọng số theo **khoảng cách không gian**, nên nó không phân biệt được điểm ảnh nhiễu với điểm ảnh nằm trên một biên thật — kết quả là biên bị làm mờ cùng với nhiễu.

Lọc song phương [11] nhân thêm một nhân trọng số theo **chênh lệch cường độ**:

$$I'(\mathbf{p}) = \frac{1}{W_p} \sum_{\mathbf{q} \in S} G_{\sigma_s}(\|\mathbf{p} - \mathbf{q}\|) G_{\sigma_r}(|I(\mathbf{p}) - I(\mathbf{q})|) I(\mathbf{q})$$

Điểm ảnh lân cận có cường độ khác xa điểm trung tâm — tức nằm bên kia một biên — nhận trọng số gần bằng không, nên **biên được bảo toàn** trong khi nhiều bên trong vùng đồng nhất vẫn bị san phẳng.

Lý do chọn lọc song phương ở đây rất cụ thể: các cặp ký tự đồng hình như 8 và B, 0 và D chỉ phân biệt được nhờ **một nét biên duy nhất**. Một bước khử nhiễu làm mờ biên sẽ trực tiếp tạo ra chính loại lỗi mà toàn bộ khối hậu xử lý ở mục 3.6 sinh ra để sửa.

2.2.4. Phóng đại và nội suy

Vùng biên do bộ phát hiện cắt ra thường chỉ cao **20–40 điểm ảnh**, trong khi mô-đun nhận dạng chuẩn hoá mọi ảnh đầu vào về chiều cao cố định **48 điểm ảnh**. Nếu đưa thẳng ảnh 20 điểm ảnh vào, mô-đun sẽ tự phóng đại bằng phép nội suy mặc định của nó. Đồ án chủ động phóng đại về **64 điểm ảnh** trước, để bước nội suy nằm trong tầm kiểm soát và diễn ra **trước** các bước tăng cường tương phản thay vì sau.

Thứ tự này quan trọng: phóng đại rồi mới áp CLAHE thì CLAHE làm việc trên lưới ô có đủ điểm ảnh để lược đồ mang ý nghĩa thống kê; làm ngược lại thì lưới 8×8 trên một ảnh cao 20 điểm ảnh cho mỗi ô chưa tới 3 hàng điểm ảnh.

2.2.5. Không gian màu HSV cho phân tích màu nền

Phân loại màu nền trong không gian RGB rất kém ổn định, vì ba kênh RGB **cùng thay đổi** khi độ sáng thay đổi: một biển vàng trong bóng râm và cùng biển đó dưới nắng cho hai bộ ba RGB rất khác nhau. Không gian **HSV** tách riêng **sắc độ (H)** khỏi **độ bão hoà (S)** và **độ sáng (V)**, nên sắc độ của biển vàng gần như không đổi theo điều kiện chiếu sáng — chỉ V thay đổi.

Đồ án vì vậy phân loại màu bằng cách thống kê tỉ lệ điểm ảnh rơi vào từng dải sắc độ và chọn dải chiếm ưu thế (chi tiết cài đặt ở mục 3.5).

2.2.6. Băm tri giác cho khử trùng lặp bộ dữ liệu

Bộ dữ liệu của đồ án hợp nhất từ nhiều nguồn công khai, mà các nguồn này **fork lẫn nhau**. Nếu một ảnh nằm ở tập huấn luyện dưới tên bộ này và ở tập kiểm thử dưới tên bộ khác thì **độ chính xác đo được đang đo khả năng ghi nhớ**, không phải khả năng tổng quát hoá.

So khớp theo mã băm mật mã (MD5, SHA) không dùng được, vì chỉ cần nén lại ảnh ở chất lượng khác là mã băm đổi hoàn toàn. Cần một hàm băm mà **ảnh giống nhau về mặt thị giác cho mã băm gần nhau**.

Băm tri giác dựa trên biến đổi cosine rời rạc (pHash) [14] hoạt động theo bốn bước: đưa ảnh về thang xám và kích thước 32×32 ; áp biến đổi cosine rời rạc hai chiều; **giữ lại khối 8×8 ở góc trên trái**, tức các hệ số **tần số thấp** mô tả cấu trúc tổng thể và loại bỏ chi tiết tần số

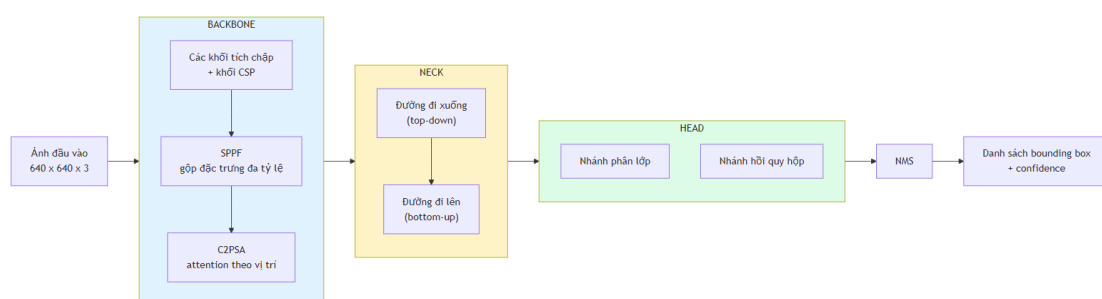
cao; so từng hệ số với trung vị của khối để sinh **64 bit**. Khoảng cách giữa hai ảnh là **khoảng cách Hamming** giữa hai mã băm.

Việc giữ lại tần số thấp chính là điều khiến pHash bền vững trước nén, đổi kích thước và thay đổi độ sáng nhẹ — và cũng chính là **giới hạn của nó**, phân tích ở mục 3.2.3.

2.3. Phát hiện vùng biến số

2.3.1. Kiến trúc một giai đoạn và ý nghĩa của anchor-free

Họ hai giai đoạn (Faster R-CNN) sinh vùng đề xuất rồi phân loại từng đề xuất, cho độ chính xác cao nhưng độ trễ lớn; họ **một giai đoạn** (YOLO, SSD) hồi quy trực tiếp trong một lần lan truyền xuôi. Ràng buộc CPU của đồ án loại họ hai giai đoạn ngay từ đầu (Hình 2.1).



Hình 2.1. Kiến trúc tổng quát backbone – neck – head của YOLO11 (theo [7])

Từ YOLOv8, họ YOLO chuyển sang **đầu dự đoán anchor-free**, và điều này có ý nghĩa riêng với bài toán biến số. Cách tiếp cận anchor-based hồi quy theo một tập hợp mẫu được thiết kế theo phân bố của bộ dữ liệu COCO; biến số **nằm ngoài phân bố đó** — một dòng khoảng 4,7:1, hai dòng khoảng 1,4:1, hai chế độ tỉ lệ cách xa nhau. Anchor-free hồi quy **trực tiếp khoảng cách từ tâm tới bốn cạnh**, nên xử lý được cả hai chế độ bằng một cơ chế duy nhất [7].

2.3.2. Chỉ số đánh giá

Với *TP* dự đoán đúng, *FP* dự đoán sai và *FN* đối tượng bỏ sót:

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}$$

Với ALPR, **recall quan trọng hơn precision**: một biến bị bỏ sót là mất vĩnh viễn, trong khi một vùng báo nhầm sẽ bị khối hậu xử lý loại vì chuỗi không khớp cú pháp.

AP là diện tích dưới đường cong Precision–Recall; **mAP** là trung bình AP trên các lớp — bài toán này chỉ có một lớp nên mAP trùng AP. Hai biến thể **không được so sánh chéo với nhau**: $\text{mAP}@0,5$ tính tại một ngưỡng IoU cố định 0,5, còn $\text{mAP}@0,5:0,95$ lấy trung bình trên mười ngưỡng từ 0,50 đến 0,95. Vì $\text{mAP}@0,5$ là số hạng lớn nhất trong mười số hạng của phép trung bình, trên cùng một mô hình và cùng một tập dữ liệu ta **luôn có** $\text{mAP}@0,5:0,95 \leq \text{mAP}@0,5$.

Khoảng cách giữa hai chỉ số này với biến số thường rất lớn, do hộp bao bọc khiến một sai lệch nhỏ theo chiều cao làm IoU tụt nhanh. Đây là lý do đề án lấy $\text{mAP}@0,5$ làm chỉ tiêu chính nhưng vẫn báo cáo $\text{mAP}@0,5:0,95$: chỉ số thứ hai mới phản ánh **độ khít của vùng cắt** đưa sang bước nhận dạng.

2.4. Nhận dạng ký tự và điểm suy giảm trên văn bản hai dòng

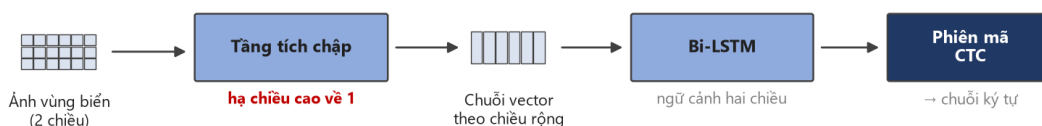
2.4.1. Kiến trúc CRNN và hàm mất mát CTC

CRNN [13] gồm ba tầng: tầng tích chập trích đặc trưng và — điểm mấu chốt — **hạ chiều cao bản đồ đặc trưng về 1**, biến ảnh thành một **chuỗi vector theo chiều rộng**; tầng hồi quy mô hình hoá ngữ cảnh; tầng phiên mã giải chuỗi đó thành văn bản.

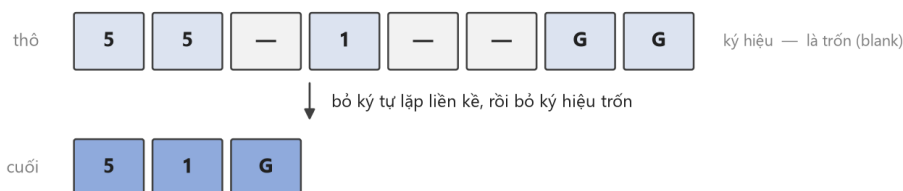
Hàm mất mát CTC [12] giải bài toán: biết chuỗi nhãn đúng nhưng **không biết mỗi ký tự nằm ở cột đặc trưng nào**. CTC thêm ký hiệu trống ε , định nghĩa ánh xạ $\mathcal{B}$ gộp ký tự lặp rồi xoá ε — ví dụ $\mathcal{B}(3\varepsilon 00\varepsilon A) = 30A$ — và tính xác suất chuỗi nhãn $\mathbf{l}$ bằng tổng xác suất **mọi** đường đi thô ánh xạ về nó:

$$p(\mathbf{l} | \mathbf{x}) = \sum_{\pi \in \mathcal{B}^{-1}(\mathbf{l})} \prod_{t=1}^T y_{\pi_t}^t$$

Ưu điểm quyết định: **không cần nhãn vị trí từng ký tự**, lý do CTC là mặc định của hầu hết bộ nhận dạng ký tự mã nguồn mở (Hình 2.2).



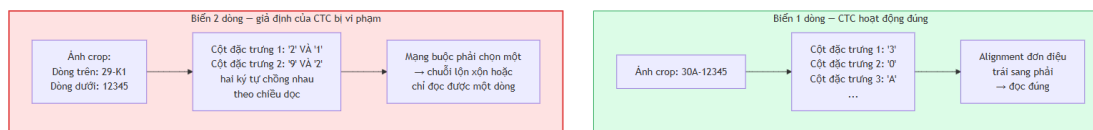
Cách CTC gộp chuỗi thô — ví dụ đọc biển 51G



Nhờ ký hiệu trống, CTC không cần nhãn vị trí từng ký tự — chỉ cần chuỗi nhãn đúng. Đó là lý do nó thành chuẩn cho nhận dạng chuỗi, và cũng là lý do nó gặp khó với văn bản nhiều dòng (mục 2.4.3).

Hình 2.2. Kiến trúc CRNN và cách CTC gộp chuỗi thô

2.4.2. Vì sao CTC gây trên biển hai dòng



Hình 2.3. Cơ chế sụp đổ của CTC trên ảnh văn bản hai dòng

Phép hạ chiều cao về 1 ở mục 2.4.1 **giả định toàn bộ văn bản nằm trên một dòng ngang** (Hình 2.3). Với ảnh hai dòng, mọi ký tự của dòng trên và dòng dưới bị **chiếu chồng lên nhau** vào cùng một cột đặc trưng, và giả định căn chỉnh đơn điệu giữa cột ảnh và chuỗi ký tự — nền tảng của CTC — không còn đúng. Hệ quả quan sát được là mô hình đọc theo thứ tự không xác định, ghép lẫn hai dòng, hoặc bỏ sót hẳn một dòng.

Vấn đề còn bị khuếch đại bởi một ràng buộc kích thước. Mô-đun nhận dạng chuẩn hoá mọi ảnh về chiều cao **48 điểm ảnh**. Biển xe mô tô có tỉ lệ khung hình khoảng 1,36, nên sau chuẩn hoá **mỗi hàng ký tự chỉ còn khoảng 24 điểm ảnh** — thấp hơn ngưỡng mà nét chữ còn tách rời được.

Hai quan sát này dẫn thẳng tới giải pháp xử lý ảnh ở mục 3.4: nếu vấn đề là *ảnh có hai dòng*, thì **biến nó thành ảnh một dòng trước khi đưa vào mô hình** — và làm sao cho hàng ký tự duy nhất đó nhận trọn ngân sách 48 điểm ảnh thay vì phải chia đôi.

2.4.3. Chỉ số CER và độ chính xác mức chuỗi

CER dựa trên khoảng cách Levenshtein, với S ký tự thay thế, D bị xoá, I chèn thừa và N tổng ký tự nhãn thật:

$$\text{CER} = \frac{S + D + I}{N}$$

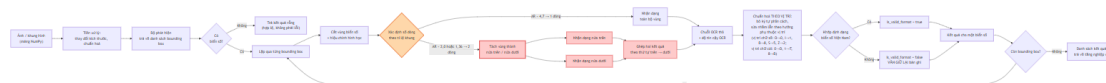
CER **có thể vượt 1** khi chuỗi dự đoán dài hơn nhãn thật rất nhiều — đúng tình huống CTC gặp ảnh hai dòng.

Độ chính xác mức chuỗi đếm số biển có **toàn bộ** chuỗi khớp chính xác. Đây là chỉ số phản ánh đúng giá trị sử dụng, và quan hệ của nó với CER **bất lợi một cách không tuyến tính**: với biển 8 ký tự, nếu xác suất đọc đúng mỗi ký tự là p thì xác suất đúng cả chuỗi là p^8 . Với $p = 0,99$ con số này chỉ còn khoảng 0,923; với $p = 0,95$ nó tụt xuống khoảng 0,663. Đây là lý do một bộ nhận dạng có CER rất tốt trên văn bản tài liệu vẫn có thể thất bại trên biển số.

CHƯƠNG 3. THIẾT KẾ VÀ CÀI ĐẶT

3.1. Kiến trúc tổng thể

Toàn bộ luồng xử lý của đường ống nhận dạng trình bày ở Hình 3.1.



Hình 3.1. Luồng xử lý của đường ống nhận dạng; các khối tô đỏ là nhánh biến hai dòng

Đường ống gồm bốn khối nối tiếp, và độ chính xác cuối cùng là **tích** của độ chính xác từng khối — một khối yếu kéo cả chuỗi xuống:

1. **Phát hiện vùng biển** — YOLO11n trên ảnh đầu vào, trả về danh sách hộp bao;
2. **Xử lý ảnh vùng biển** — cắt, phân loại bố cục theo hình học, tách và ghép, rồi phóng đại, tăng cường tương phản và khử nhiễu (mục 3.4);
3. **Nhận dạng ký tự** — PaddleOCR trên dải ảnh một dòng đã chuẩn bị;
4. **Hậu xử lý theo quy chuẩn** — chuẩn hoá chuỗi theo bộ luật ràng buộc vị trí (mục 3.6).

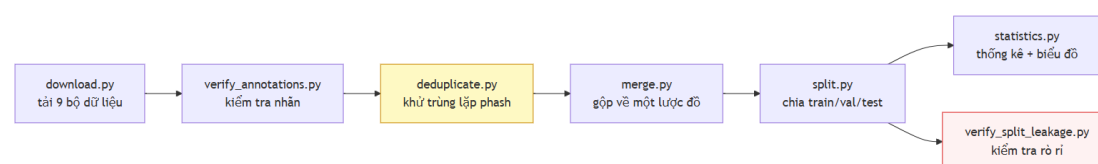
Khối 2 và khối 4 phần do đề án tự thiết kế; khối 1 và khối 3 dùng mô hình có sẵn.

Toàn bộ đường ống được đóng gói thành một **gói Python độc lập không phụ thuộc tầng web**. Ràng buộc này không phải hình thức: nó cho phép cùng một mã chạy được trong sổ tay thử nghiệm, trong kịch bản đo đạc và trong dịch vụ đang vận hành. Bài học ngược lại đã xảy ra trong quá trình thực hiện đề án: một kịch bản đo **chép lại** các bước của đường ống thay vì **gọi** nó, nên mỗi bước mới thêm vào đường ống đều rơi ra ngoài phép đo, và số liệu công bố mô tả một hệ thống ngắn hơn hệ thống thực tế.

Ba lớp trừu tượng có hợp đồng thống nhất: lớp phát hiện trả danh sách vùng biển đã lọc ngưỡng và khử chồng lấn, **danh sách rỗng là kết quả hợp lệ chứ không phải lỗi**; lớp nhận dạng trả chuỗi thô kèm độ tin cậy, **không** tự sửa lỗi ký tự; lớp chuẩn hoá trả về cả chuỗi không hợp lệ kèm cờ đánh dấu. Chính việc lớp nhận dạng không được phép tự sửa lỗi là điều kiện để **đo tách bạch** đóng góp của khối hậu xử lý ở mục 4.3.2.

3.2. Xây dựng bộ dữ liệu

3.2.1. Đường ống sáu bước xây dựng bộ dữ liệu



Hình 3.2. Đường ống sáu bước xây dựng bộ dữ liệu

Hình 3.2 tóm tắt sáu bước; mỗi bước là một kịch bản độc lập sinh báo cáo dạng dữ liệu có cấu trúc; một kịch bản điều phối chạy toàn chuỗi bằng một lệnh. Kết quả: **15.133 ảnh** hợp

nhất từ **7 bộ công khai**, sau khi loại **11.978 ảnh trùng lặp (44,2%)** từ **27.111 ảnh** ban đầu. Chia theo tỉ lệ 70/20/10 thành **10.592 / 3.027 / 1.514 ảnh** (Bảng 3.1).

Bảng 3.1. Đóng góp của từng bộ dữ liệu trước và sau khử trùng lặp

#	Bộ dữ liệu	Vào hợp nhất	Còn lại	Bị loại
1	roboflow_school_fuhih	8.357	6.868	17,8%
2	hf_vn_plates_segment	4.578	4.375	4,4%
3	roboflow_traffic_camera	3.843	3.162	17,7%
4	roboflow_eric_nguyen	840	353	58,0%
5	roboflow_demo_tracking	236	235	0,4%
6	roboflow_cuong_ta	8.254	140	98,3%
7	roboflow_tran_ngoc_xuan_tin	1.005	0	100%
	Tổng †	27.111	15.133	44,2%

† Cột *Vào hợp nhất* cộng lại được 27.113; bước hợp nhất gạt hai ảnh nằm trong danh sách loại trừ ra trước khi ghi (skipped_excluded = 2 trong báo cáo hợp nhất), nên tổng thực ghi là 27.111.

3.2.2. Khử trùng lặp chéo bộ

Bảng trên là lý do bước này bắt buộc phải có. Một bộ vào hợp nhất với **1.005 ảnh và ra với 0 ảnh** — toàn bộ nội dung của nó đã có sẵn trong các bộ khác. Hệ quả trực tiếp: **không được cộng dồn số ảnh công bố của từng bộ để suy ra quy mô thật**.

Vết cạn mọi cặp trong 27.111 ảnh là khoảng 367 triệu phép so sánh, không khả thi. Kịch bản dùng **băm đa chỉ mục**: cắt mã băm 64 bit thành ngưỡng + 1 dải. Theo nguyên lý chuồng bồ câu, hai mã băm khác nhau **tối đa** ngưỡng bit bắt buộc phải trùng khớp hoàn toàn trên **ít nhất một dải**. Tập ứng viên thu được vì vậy chứa **mọi** cặp thật, rồi được xác minh lại bằng khoảng cách Hamming chính xác — thuật toán là **chính xác, không xấp xỉ**.

Bước chia tập giữ **mọi thành viên của một nhóm trùng lặp trong cùng một tập con**, nên những bản trùng không bị xoá cũng không thể rò rỉ giữa tập huấn luyện và tập kiểm thử.

3.2.3. Giới hạn của băm tri giác: nó tóm tắt khung ảnh, không tóm tắt chiếc xe

Đây là **giới hạn không khắc phục được** bằng công cụ hiện có, và đồ án ghi nhận thay vì bỏ qua.

Một lần kiểm tra độc lập ở ngưỡng Hamming 10 trên phiên bản đầu của bộ dữ liệu tìm thấy **619 cặp gần trùng giữa tập huấn luyện và tập kiểm thử**; kiểm bằng mắt cho thấy đó là **cùng một chiếc xe, cùng chuỗi biển số, xuất hiện ở cả hai tập**. Đường ống không bắt được vì bước chia tập gom nhóm ở ngưỡng 5 và lần kiểm tra đầu cũng đo lại ở đúng ngưỡng 5 — một **lập luận vòng tròn**: đo ở ngưỡng đã dùng để gộp thì chỉ chứng minh bước gộp đã chạy đúng đặc tả, không chứng minh thêm điều gì.

Nâng ngưỡng cũng không giải quyết được, và lý do nằm ngay ở cơ chế của pHash trình bày ở mục 2.2.6. Mã băm mô tả **cấu trúc tần số thấp của toàn khung ảnh**, nên hai chiếc xe khác nhau đi qua **cùng một camera** có khoảng cách Hamming rất nhỏ — vì 90% khung hình (mặt đường, vạch kẻ, nền) giống hệt nhau. Đánh đổi vì vậy không thoát được:

- **ngưỡng thấp** bỏ sót các cặp "cùng xe, khác ngày";
- **ngưỡng cao** gộp nhầm hàng nghìn ảnh xe khác nhau chụp cùng một camera.

Số liệu xác nhận: ở ngưỡng 12 còn **791 cặp**, ở ngưỡng 15 là **3.529 cặp**, ở ngưỡng 20 lên **137.506 cặp** — mức mà phần lớn đã là dương tính giả.

Kết luận trung thực: đồ án khẳng định được *bước khử trùng lặp ở ngưỡng 10 đã chạy đúng đặc tả*, và **không** khẳng định được *tập kiểm thử độc lập với tập huấn luyện*. Rò rỉ ở mức ngữ nghĩa cần so khớp theo chuỗi biến số hoặc đặc trưng phương tiện mới phát hiện được. Mọi chỉ số ở mục 4.2 vì vậy phải đọc như **cận trên lạc quan**.

3.3. Huấn luyện bộ phát hiện

Cấu hình lượt huấn luyện chính thức được trích từ tệp tham số do thư viện tự sinh — bản ghi *đã thực thi* chứ không phải *dự định* (Bảng 3.2).

Bảng 3.2. Siêu tham số lượt huấn luyện chính thức

Tham số	Giá trị	Lý do
Mô hình khởi đầu	YOLO11n tiền huấn luyện COCO, 2.590.035 tham số	Biến thể nhỏ nhất, do ràng buộc CPU
Độ phân giải đầu vào	640	Đúng theo chỉ tiêu; cũng là độ phân giải mọi số liệu tốc độ CPU chính thức được đo
Số epoch · kích thước lô	20 · 8	Ngân sách thời gian CPU
Thuật toán tối ưu	AdamW, tốc độ học ban đầu 0,001, lịch cosine	
Hạt giống ngẫu nhiên	42 , kèm chế độ tắt định	Chỉ chạy được một lượt nên ít nhất phải tái lập được
Lật ngang	Tắt hoàn toàn	Lệch có chủ ý so với mặc định — xem dưới

Việc **tắt phép lật ngang** là quyết định xử lý ảnh đáng chú ý nhất trong cấu hình này. Lật ngang là phép tăng cường dữ liệu mặc định và hữu ích trong hầu hết bài toán phát hiện, nhưng ở đây nó sinh ra **ký tự đối xứng gương** — một phân bố không bao giờ xuất hiện trong thực tế. Giữ nó lại là dạy mô hình một bất biến mà bài toán không có.

Chi phí: 30,2 phút mỗi epoch, tổng **36.181 giây tương đương 10,05 giờ** liên tục trên CPU. Chi phí này khiến **tìm kiếm siêu tham số bất khả thi**: đồ án báo cáo *một* cấu hình huấn luyện, không phải kết quả của một quá trình tối ưu, và mọi chỉ số là kết quả **một lần chạy** không có khoảng tin cậy.

3.4. Khối xử lý ảnh vùng biên số

Đây là phần trọng tâm của đồ án. Đầu vào là vùng ảnh do bộ phát hiện cắt ra; đầu ra là một **dải ảnh một dòng** đưa sang bộ nhận dạng.

3.4.1. Chuỗi bước và nguyên tắc bất tất độc lập

Mọi bước trong mục này đều **bất tất được độc lập** qua biến cấu hình — điều kiện để chương 4 **bóc tách đóng góp của từng bước**. Thứ tự trên **đường chạy chính**:

cắt vùng → ước lượng số dòng → (*nếu hai dòng*) tách hai nửa → ghép ngang → phóng đại → thang xám → CLAHE → lọc song phương → nhận dạng

Ước lượng số dòng chạy trên vùng cắt thô (các bước tăng cường phía sau không làm đổi tỉ lệ khung hình); tiền xử lý chạy trên dải ảnh **đã ghép** để CLAHE cân bằng tương phản trên một hàng ký tự liền mạch; còn bước nắn hình không nằm trên đường chạy chính — nó thuộc bậc thang thử lại ở mục 3.4.6.

1 · Vùng cắt từ bộ phát hiện 2 · Ước lượng số dòng



83 × 74 px · tỉ lệ 1.12



tỉ lệ 1.12 < 2,50 → hai dòng

3 · Tách hai nửa có chồng lấn



trên 83×30 · dưới 83×50 px

4 · Ghép ngang



221 × 50 px · tỉ lệ 4.42 · một hàng ký tự nhận tròn 50 px chiều cao

5 · Phóng đại và chuyển thang xám



cao 64 px · 3 kênh → 1

6 · CLAHE



giới hạn tương phản 2,0 · lưới ô 8 × 8 · Khử vùng chói lóa cục bộ

7 · Lọc song phương



khử nhiễu mà giữ biên · Bảo toàn biên nét ký tự trước OCR

Kết quả nhận dạng: 59K1-201.73

Hình 3.3. Toàn bộ chuỗi xử lý trên một biển thật, ảnh chụp sau từng bước

Hình 3.3 là kết quả chạy **chính các hàm của mô hình đề xuất**: mỗi khung là mảng ảnh thật ở đầu ra của bước tương ứng. Khung 3 và khung 4 cho thấy trực tiếp điều cả mục này mô tả bằng chữ — **tỉ lệ khung hình nhảy từ 1,12 lên 4,42**, hai hàng ký tự cao 30 px thành một hàng duy nhất nhận trọn 50 px. Phần chân hàng trên lọt vào nửa dưới ở khung 3 không phải lỗi cắt mà là vùng chồng lấn của mục 3.4.4.

3.4.2. Tiền xử lý

Ba bước độc lập, cơ sở lý thuyết ở mục 2.2:

- **Chuyển thang xám**, vì ký tự không mang thông tin phân biệt trong kênh màu.
- **CLAHE**, hệ số giới hạn **2,0** trên lưới ô **8 × 8**, để xử lý mảng chói cục bộ do bề mặt phản quang.
- **Lọc song phương** thay cho làm mờ Gauss, để khử nhiễu mà không phá biên — yếu tố quyết định phân biệt các cặp ký tự đồng hình.

Vùng biển được **phóng đại về 64 điểm ảnh** chiều cao trước toàn bộ chuỗi trên, vì ảnh do bộ phát hiện sinh ra thường chỉ cao 20–40 điểm ảnh.

3.4.3. Ước lượng số dòng bằng tỉ lệ khung hình

Số dòng suy từ tỉ lệ chiều rộng trên chiều cao của vùng biển, **ngưỡng phân loại 2,5**: tỉ lệ nhỏ hơn ngưỡng được xếp vào nhóm hai dòng.

Ngưỡng này là **đề xuất của đồ án, không phải quy định pháp lý**. Quy chuẩn chỉ cung cấp ba tỉ lệ vật lý $4,727 \cdot 2,000 \cdot 1,357$ (mục 2.1.2), để lại khoảng trống rộng giữa 2,000 và 4,727. Giá trị 2,5 được đặt **lệch hẳn về phía nhóm hai dòng** thay vì đặt ở giữa khoảng trống, và lý do là **tính bất đối xứng của chi phí sai sót**: đường xử lý hai dòng **suy giảm êm** khi gặp đầu vào một dòng — nó chỉ tách một ảnh vốn đã một dòng thành hai nửa rồi ghép lại, kết quả gần như không đổi — trong khi chiều ngược lại thì không, một biển hai dòng đi thẳng vào bộ nhận dạng sẽ hỏng theo cơ chế ở mục 2.4.2.

Dài 2,5–3,0 vẫn là **vùng bất định**, và nó bất định theo cả hai chiều: một biển một dòng chụp nghiêng lớn có thể cho tỉ lệ rơi xuống khoảng này, còn một biển hai dòng nghiêng thì có tỉ lệ **vọt lên trên** ngưỡng và đi nhầm sang nhánh một dòng.

Đường chạy chính **không có cách nào tự phát hiện** mình vừa phân loại nhầm — nó chỉ đo một con số và so với một ngưỡng. Đây chính là lý do bậc thang thử lại ở mục 3.4.6 tồn tại: nó không sửa ngưỡng mà **dùng kết quả đọc hỏng làm tín hiệu** để nắn hình rồi phân loại lại.

3.4.4. Tách hai nửa có chồng lấn

Vùng biển được cắt thành hai nửa theo chiều dọc, nhưng **hai nửa cố ý chồng lên nhau**: nửa trên kết thúc tại **5/12** chiều cao, nửa dưới bắt đầu tại **1/3**, tạo vùng chồng lấn bằng **1/12** chiều cao biển.

Căn cứ kỹ thuật của hai phân số 5/12 và 1/3 xuất phát trực tiếp từ các tham số kích thước quy định tại QCVN 08:2024/BCA [5]: trên biển số xe mô tô hai dòng, seri đăng ký ở dòng

trên có kích thước chữ cái và chữ số tương đương dòng dưới, nhưng khoảng cách từ mép trên đến đỉnh ký tự dòng trên thường nhỏ hơn khoảng cách từ đáy số dòng dưới đến mép dưới biển. Phép cắt nửa trên tại 5/12 chiều cao biển đảm bảo thu trọn toàn bộ thân chữ của seri; trong khi nửa dưới bắt đầu tại 1/3 chiều cao biển đảm bảo ôm trọn cụm số thứ tự ngay cả khi biển số bị dập lệch nhẹ. Khoảng cách giao nhau 1/12 chiều cao đóng vai trò như một vùng đệm an toàn: nó triệt tiêu hoàn toàn rủi ro cắt đứt nét chân của chữ dòng trên hoặc cắt mất phần mũ của số dòng dưới.

Thiết kế xuất phát từ tính bất đối xứng của chi phí sai sót, giống mục 3.4.3 nhưng ở một đại lượng khác. Cắt phạm vào nét đáy ký tự dòng trên hoặc đỉnh của ký tự dòng dưới **phá huỷ thông tin không phục hồi được** — một 8 bị cắt chân thành 9 hoặc 0 là lỗi vĩnh viễn. Ngược lại, để lọt vài hàng điểm ảnh của nửa còn lại chỉ tạo ra một dải nhiễu mà bộ nhận dạng xử lý như nền.

Vùng chồng lấn này về sau hoá ra còn có một tác dụng thứ hai mà thiết kế ban đầu không lường trước, phân tích ở mục 3.4.7.

3.4.5. Ghép ngang

Hai nửa được ghép theo chiều ngang bằng phép nối mảng hstack, **nửa trên đặt bên trái** để bảo toàn thứ tự đọc. Chiều cao chung lấy bằng giá trị lớn nhất trong ba đại lượng: chiều cao nửa trên, chiều cao nửa dưới, và **48 điểm ảnh** — đúng bằng chiều cao đầu vào cố định của mô-đun nhận dạng.

Chi tiết cuối cùng là điểm mấu chốt của toàn bộ thiết kế. Sau khi ghép, ảnh chỉ còn **một hàng ký tự duy nhất**, và hàng đó nhận **trọn ngân sách 48 điểm ảnh** thay vì hai hàng chia nhau mỗi hàng 24 điểm ảnh. Phép biến đổi này vô hiệu hoá **đồng thời cả hai nguyên nhân** đã phân tích ở mục 2.4.2: giả định căn chỉnh đơn điệu của CTC lại đúng, và độ phân giải mỗi hàng ký tự tăng gấp đôi.

Đây là một minh hoạ trực tiếp cho luận điểm của môn học: bài toán không được giải bằng cách thay mô hình mạnh hơn, mà bằng cách **biến đổi ảnh đầu vào cho khớp giả định của mô hình sẵn có**.

3.4.6. Nắn hình và bậc thang thử lại

Biến chụp nghiêng vừa làm ký tự biến dạng phối cảnh, vừa làm **hộp bao nở rộng theo chiều ngang** khiến tỉ lệ khung hình đo được vượt ngưỡng 2,5 của mục 3.4.3: biến hai dòng bị phân loại nhầm thành một dòng, không được tách và trả về chuỗi rỗng — lỗi **đi nhầm nhánh xử lý**, không phải "đọc kém đi". Hai phép hiệu chỉnh được cài để kéo vùng biển về đúng nhánh:

- **Nắn hình** — nhị phân hoá bằng Otsu ở cả hai cực, lấy vùng liên thông lớn nhất, khớp một hình chữ nhật xoay, xoay cho cạnh dài nằm ngang rồi **cắt lại sát**; tỉ lệ của phần cắt sát mới là hình dạng thật của biển — đúng đại lượng mà bước phân loại cần.
- **Giãn theo chiều dọc** — cho biển bị nén do chụp chếch từ trên xuống, khi không có góc xoay nào để nắn; giá trị nằm ở việc **định tuyến lại**, không ở chi tiết ảnh nội suy thêm.

Hai phép này tổ chức thành một **bậc thang thử lại**, chỉ kích hoạt sau khi lượt đọc đầu tiên trượt kiểm tra định dạng. Vì cổng chỉ mở khi kết quả đã không hợp lệ, tập bị can thiệp và tập đang đúng là **hai tập rời nhau** — bậc thang không thể làm hỏng một biển vốn đã đọc đúng, nên bật mặc định được. Bước nắn hình còn ba cổng an toàn tự lùi về "trả nguyên vùng cắt": góc nghiêng dưới 1,5°, góc trên 35°, hoặc vùng liên thông lớn nhất dưới 25% diện tích vùng cắt.

Chi phí và lợi ích đo được trình bày ở mục 4.4.2, kèm một quyết định **tắt** một bậc trong đó.

Cần nhấn mạnh một đánh đổi có chủ đích trong thiết kế: giải thuật nắn hình chỉ áp dụng phép xoay phẳng hai chiều (2D affine rotation) dựa trên việc khớp hình chữ nhật có hướng diện tích tối thiểu (minAreaRect) [16]. Hệ thống chủ động không tích hợp các phép biến đổi phối cảnh ba chiều (3D Homography/Perspective Transformation) phức tạp. Lý do trực tiếp là ràng buộc thời gian thực trên CPU: tính toán ma trận phối cảnh và nội suy lại toàn bộ lưới điểm ảnh trên CPU sẽ đẩy độ trễ p95 vượt trần cho phép, trong khi phép xoay phẳng 2D có chi phí tính toán xấp xỉ 0 ms nhưng vẫn giải quyết được đại đa số các trường hợp biến số bị lệch góc chụp phổ biến trong thực tế.

3.4.7. Bước phục hồi dòng trên

Chế độ hồng quan sát được: chuỗi 29E-015.66 chỉ đọc ra 015.66 — sau khi ghép, bộ phát hiện văn bản của PaddleOCR chỉ khoanh được một vùng chữ và bỏ qua nửa bên trái. Giả thuyết tự nhiên — **bỏ hẳn phép ghép, đọc riêng từng nửa rồi nối chuỗi** — được kiểm bằng thí nghiệm A/B trên 200 biển hai dòng và **bị bác bỏ dứt khoát** (mục 4.4.1): khi đọc riêng, dải chồng lấn của mục 3.4.4 bị nhận dạng **hai lần** và sinh ký tự thừa (84G122593 → 84-G124E009.01225.93), trong khi trên dải đã ghép nó nằm giữa hai cụm ký tự và bị loại như một mảnh nhiễu.

Thiết kế cuối cùng vì vậy **giữ nguyên chiến lược ghép** và chỉ bổ sung một bước phục hồi có điều kiện chặt: kích hoạt khi đồng thời (a) vùng biển phân loại hai dòng, (b) chuỗi sau chuẩn hoá không hợp lệ, (c) chuỗi thô khác rỗng — hệ thống đọc thêm một lượt trên **riêng nửa trên**, ghép với chuỗi thô rồi chuẩn hoá lại, và chỉ nhận kết quả vượt kiểm tra định dạng.

3.5. Phân loại màu nền trong không gian HSV

Mục 2.1.3 đã chỉ ra giới hạn về nguyên tắc: chuỗi ký tự không phân biệt được biển vàng với biển trắng. Bộ phân loại màu cung cấp **nguồn bằng chứng thứ hai**.

Bộ phân loại chuyển vùng biển sang không gian HSV, thống kê tỉ lệ điểm ảnh theo từng dải sắc độ và chọn dải chiếm ưu thế. Ba quyết định thiết kế đáng lưu ý:

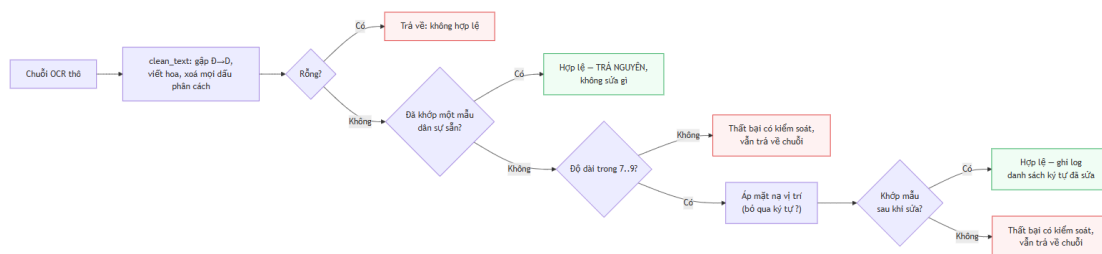
Chỉ lấy mẫu vùng trung tâm, thu biên vào 18% mỗi phía. Hộp bao do bộ phát hiện sinh ra hiếm khi ôm sát mép biển, nên rìa hộp thường chứa màu thân xe phía sau. Với biển nhỏ, phần rìa đó đủ để chiếm ưu thế và lật kết quả.

Không loại trừ điểm ảnh thuộc ký tự. Ký tự chiếm thiếu số diện tích biến, và việc bổ sung một bước phân đoạn ký tự sẽ đưa vào chuỗi xử lý một khâu **kém ổn định hơn chính khâu nó bảo vệ**.

Trả kết quả *không xác định* khi dải chiếm ưu thế không đạt 30%. Một kết luận sai về màu tương đương khẳng định một loại phương tiện không chứng minh được; thừa nhận không xác định được chỉ là ghi nhận một giới hạn.

Kết quả hợp nhất giữa hai nguồn bằng chứng tuân một **ràng buộc an toàn**: màu nền chỉ được phép **nâng cấp một ứng viên mà bộ luật ký tự đã coi là hợp lý**, và nếu phán quyết ban đầu không nằm trong tập ứng viên thì kết quả giữ nguyên. Nói cách khác, màu nền không thể tạo ra một họ biến mà bộ luật ký tự đã bác bỏ (Hình 3.4).

3.6. Bộ luật hậu xử lý theo vị trí



Hình 3.4. Thuật toán chuẩn hoá chuỗi biến số theo bộ luật ràng buộc vị trí

Khối này là thành phần do đồ án tự thiết kế hoàn toàn. Nó khai thác ba ràng buộc đặc thù đã trình bày ở mục 2.1.1.

a) Tập mã tình. Khối lưu **81 mã đang sử dụng**, song song với tập **8 mã không bao giờ được cấp**. Lưu trường minh cả hai tập cho phép kiểm thử khẳng định chúng phủ đúng dải 11–99.

b) Mặt nạ vị trí. Ba mặt nạ tương ứng ba độ dài chuỗi hợp lệ, trong đó D bắt buộc chữ số, L bắt buộc chữ cái, ? là ký tự đại diện không áp đặt kiểu:

- chuỗi 8 ký tự (ô tô, seri 5 chữ số): DDLDDDDD
- chuỗi 7 ký tự (ô tô, seri 4 chữ số kiểu cũ): DDLDDDD
- chuỗi 9 ký tự (xe máy): DDL?DDDDD

Ký tự đại diện tại **chỉ số 3** của chuỗi 9 ký tự là chi tiết thiết kế then chốt. Hai kiểu biến xe máy cùng 9 ký tự nhưng khác nhau **đúng tại vị trí này**: kiểu mới dùng seri hai chữ cái, kiểu cũ dùng một chữ cái kết hợp một chữ số và vẫn lưu hành hợp pháp. Tách thành hai mặt nạ riêng sẽ buộc phải áp kiểu tại chỉ số 3, và kết quả thực nghiệm thực tế cho thấy khi đó **một trong hai kiểu bị phá huỷ**. Đây là vị trí duy nhất trong toàn hệ thống biến số Việt Nam mà cả chữ cái lẫn chữ số đều hợp lệ.

c) Bảng ánh xạ nhầm lẫn và tính bất đối xứng. Hai bảng riêng biệt được áp tại vị trí bắt buộc chữ số và vị trí bắt buộc chữ cái, và phát hiện trung tâm là **hai bảng bất đối xứng**:

- Chữ cái O → số 0 (O → 0) tại vị trí chữ số là hợp lý;

- Số 0 $\rightarrow$ chữ cái O ($0 \rightarrow O$) **không bao giờ** hợp lý, vì chữ O không thuộc tập seri hợp lệ.

Do cả chữ cái O và Q đều bị loại trừ, ứng viên đồng hình duy nhất còn lại tại vị trí chữ cái là D, nên chiều đúng là $0 \rightarrow D$. Ký tự R **không được ánh xạ trong mọi trường hợp**, vì nó hợp lệ tại vị trí seri thứ hai của biển xe máy. Nguyên tắc an toàn: ký tự không có mục trong bảng thì giữ nguyên.

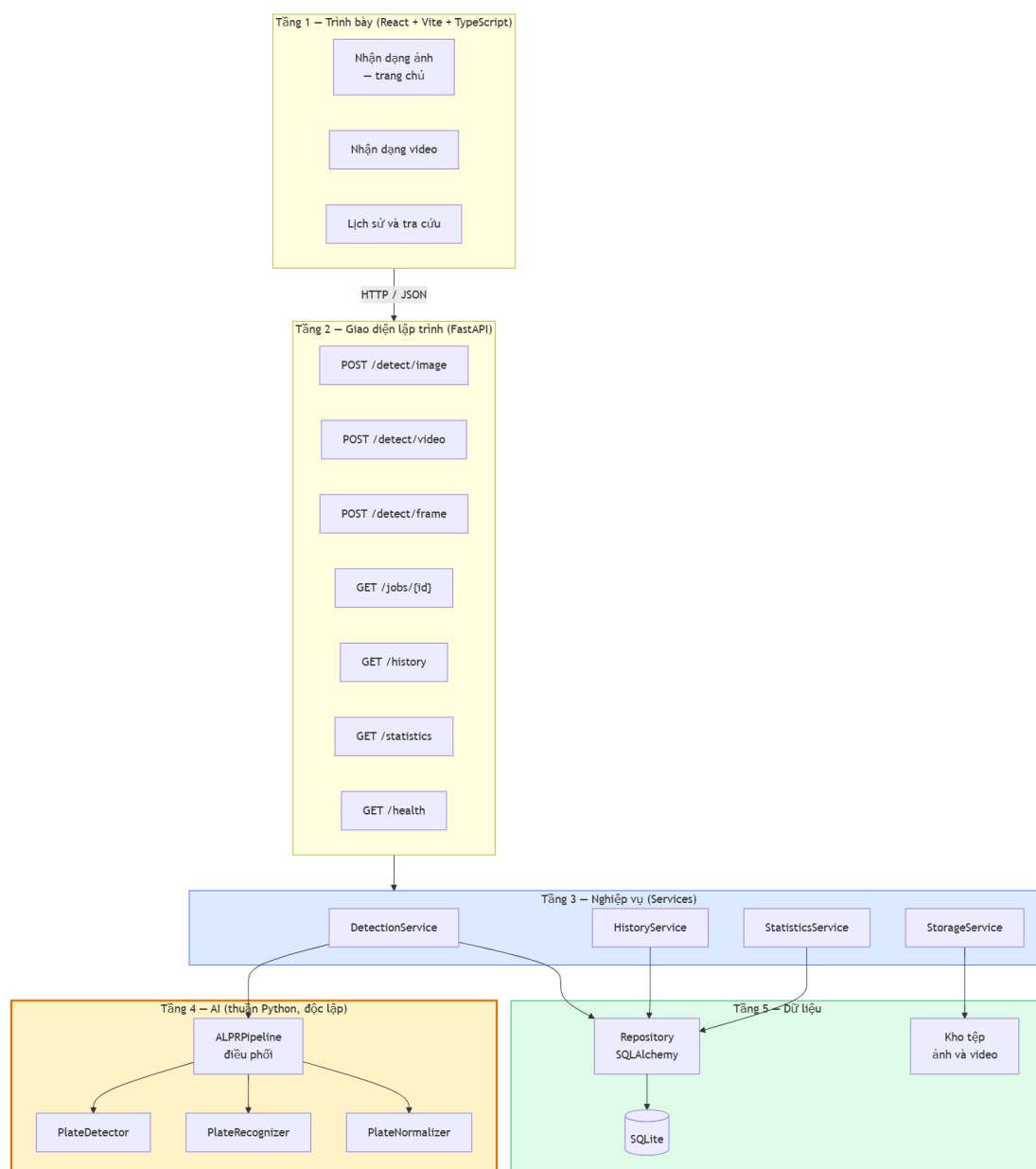
Cần nêu rõ một giới hạn: hai bảng này **suy từ lập luận hình dạng ký tự chứ không từ đo đạc**, và một số cặp mang tính phỏng đoán. Mục 4.3.3 đối chiếu chúng với ma trận nhằm lần đo được.

d) Thuật toán. Ba nguyên tắc: **thứ biểu thức chính quy trước khi sửa bất cứ thứ gì**, vì với chuỗi vốn đã hợp lệ thì mọi can thiệp chỉ có thể làm sai đi; **không chuỗi nào bị loại bỏ** — chuỗi không sửa được vẫn trả về kèm cờ không hợp lệ; và **chuỗi thô được giữ song song với chuỗi đã sửa**.

Nguyên tắc thứ ba có hệ quả trực tiếp lên cơ sở dữ liệu: hai cột riêng cho chuỗi thô và chuỗi đã chuẩn hoá cùng tồn tại trong lược đồ. Đây là **điều kiện cần để phép đo ở mục 4.3.2 thực hiện được**, và nó phải có mặt từ giai đoạn thiết kế chứ không thể bổ sung về sau.

3.7. Ứng dụng trình diễn

Phần này không phải trọng tâm của môn học nên chỉ nêu những quyết định có liên quan tới khối xử lý ảnh (Hình 3.5).



Hình 3.5. Kiến trúc phân tầng và chiều phụ thuộc

Hệ thống gồm **máy chủ FastAPI** phục vụ mười thao tác HTTP trên chín đường dẫn, **cơ sở dữ liệu SQLite** lưu lịch sử nhận dạng, **giao diện web React** bốn trang (nhận dạng ảnh, nhận dạng video, quét webcam trực tiếp, tra cứu lịch sử), và **đóng gói Docker Compose** khởi động toàn bộ bằng một lệnh.

Ba chi tiết đáng ghi nhận:

Ảnh không chứa biển số trả mã thành công kèm danh sách rỗng, không phải mã lỗi. Kết quả nhận dạng vẫn tồn tại và là tập rỗng; trả mã lỗi sẽ loại toàn bộ trường hợp âm khỏi thống kê. Đây là cùng một quyết định đã áp ở tầng suy luận (mục 3.1).

Giao diện hiển thị đồng thời chuỗi thô và chuỗi đã chuẩn hoá khi hai chuỗi khác nhau.
Điều này biến một cột dữ liệu phục vụ nghiên cứu thành **bằng chứng quan sát được ngay trong lúc trình diễn**: người xem thấy trực tiếp khối hậu xử lý vừa sửa gì.

Yêu cầu xử lý video trả mã *đã tiếp nhận* thay vì mã thành công, vì một video 60 giây, với bước lấy mẫu 5 khung hình, cần khoảng một đến ba phút xử lý trên CPU tùy độ trễ từng khung, và không client nào chờ được.

CHƯƠNG 4. THỰC NGHIỆM VÀ ĐÁNH GIÁ

4.1. Môi trường, dữ liệu và quy ước

Môi trường. Mọi số liệu **hiệu năng** ở mục 4.5 đo trên một máy trạm duy nhất, **Intel Core i5-14600K (14 nhân / 20 luồng)**, **không có GPU CUDA**; toàn bộ suy luận và huấn luyện chạy trên CPU. Cần nêu số nhân chứ không chỉ nêu "trên CPU", vì độ trễ tỉ lệ trực tiếp với nó và hệ thống đặt cứng số luồng tính toán. Ngược lại, số liệu **độ chính xác** không phụ thuộc phần cứng: cùng mô hình và cùng dữ liệu thì máy nào cũng cho kết quả đó. Phiên bản hệ điều hành, thư viện và Python ghi ở Phụ lục A.

Giao thức đo. Trọng số được **đóng băng trước** mọi phép đo; tập kiểm thử **không được chạm vào** trong huấn luyện lần khi chọn epoch. Khi đo độ trễ: kích thước lô bằng 1, bỏ 3 lượt khởi động nóng, báo cáo **p50 / p95 / p99 chứ không báo cáo trung bình** — trung bình che mất đuôi phân bố, mà chỉ tiêu lại phát biểu theo p95.

Hai tập đánh giá, hai mẫu số khác nhau. Chỉ số của bộ phát hiện đo trên **tập kiểm thử 1.514 ảnh / 1.611 đối tượng**. Chỉ số nhận dạng chỉ đo được trên **tập con có nhãn chuỗi ký tự — 2.801 biển**, vì phần lớn ngữ liệu chỉ có nhãn hộp bao. Mẫu số nhỏ này là một hạn chế thật, ghi ở mục 4.7.

Quy ước viết tắt. Ba đại lượng dùng lại nhiều lần. (*Lưu ý: các mã **E1–E6** ở mục 4.6 là mã loại lỗi, không liên quan tới ba ký hiệu này.*)

Ký hiệu	Nghĩa
C	Đúng ở mức ký tự, tức 1 – CER
S₀	Đúng cả chuỗi , đo trên chuỗi thô — trước hậu xử lý
S₁	Đúng cả chuỗi , sau hậu xử lý

4.2. Kết quả phát hiện vùng biển

4.2.1. Chỉ số tổng thể

Cả bốn chỉ tiêu của bộ phát hiện đều đạt mục tiêu, đo bằng công cụ đánh giá chuẩn của thư viện tại ngưỡng tin cậy 0,25 (Bảng 4.1).

Bảng 4.1. Kết quả phát hiện trên tập kiểm thử 1.514 ảnh

Chỉ số	Ngưỡng tối thiểu	Mục tiêu	Đo được	
mAP@0,5	0,85	0,90	0,9829	Đạt
mAP@0,5:0,95	0,55	0,65	0,7834	Đạt
Precision	0,88	0,92	0,9837	Đạt
Recall	0,85	0,90	0,9714	Đạt
F1	—	—	0,9775	—

Ba lưu ý khi đọc bảng này. **Một**, bài toán chỉ có **một lớp**, nên giá trị mAP cao là bình thường và **không phải bằng chứng về độ khó đã vượt qua**; mAP một lớp không so trực tiếp được với mAP nhiều lớp trên các bộ dữ liệu tổng quát. **Hai**, chỉ số thực sự quyết định ở đây là **mAP@0,5:0,95**, vì độ khít của hộp bao ảnh hưởng trực tiếp đến chất lượng vùng cắt đưa sang khối xử lý ảnh. **Ba**, chỉ số tổng thể **che giấu phân bố** — hai mục sau tách nó ra.

4.2.2. Tách theo bố cục biên

Bảng 4.2. Kết quả phát hiện tách theo biên một dòng và hai dòng

Chỉ số	Biên một dòng	Biên hai dòng	Chênh (điểm %)
Số đối tượng (<i>tổng 1.611</i>)	286	1.325	—
mAP@0,5	0,9884	0,9675	2,09
mAP@0,5:0,95	0,7526	0,7649	-1,23
Recall	0,9895	0,9691	2,04

Bảng 4.2 cho thấy chênh lệch giữa hai bố cục ở tầng phát hiện chỉ **2,09 điểm** — nhỏ. Con số này đáng nhớ, vì mục 4.3.2 sẽ cho thấy cùng phép tách đó ở tầng nhận dạng cho **23,07 điểm**. Kết luận: **bài toán biên hai dòng không nằm ở khâu phát hiện**.

4.2.3. Tách theo kích thước đối tượng

Mục này tồn tại vì toàn bộ bộ dữ liệu có **10,91% số hộp bao chiếm dưới 0,5% diện tích ảnh** — vượt ngưỡng chất lượng 10% mà đề án tự đặt; riêng trên tập kiểm thử tỉ lệ này là 16,3% (262/1.611). Đối tượng nhỏ là chế độ thất bại đã ghi nhận rộng rãi của bộ phát hiện một giai đoạn, nên một con số mAP tổng sẽ **giấu chế độ thất bại đó sau giá trị trung bình** (Bảng 4.3).

Bảng 4.3. Kết quả phát hiện tách theo dải kích thước hộp bao

Dải (diện tích hộp / diện tích ảnh)	Số đối tượng	mAP@0,5	mAP@0,5:0,95	Recall
Rất nhỏ — dưới 0,5%	262	0,8553	0,5249	0,8740
Nhỏ — 0,5% đến 1%	124	0,9755	0,7055	0,9758
Trung bình — 1% đến 5%	900	0,9913	0,8005	0,9922
Lớn — 5% đến 15%	297	0,9966	0,8394	0,9966
Rất lớn — trên 15%	28 †	1,0000	0,8562	1,0000

† Dòng này chỉ có 28 đối tượng, dưới ngưỡng 30 nên không có ý nghĩa thống kê và không được dùng để so sánh.

Điểm yếu duy nhất của bộ phát hiện lộ ra ở đây: dải **rất nhỏ** rớt xuống mAP@0,5 = 0,8553 và mAP@0,5:0,95 = 0,5249, tức **hộp bao vừa dễ bỏ sót vừa kém khít**. Với biến số, hộp kém khít kéo theo hậu quả dây chuyền: vùng cắt lệch làm tỉ lệ khung hình đo sai, khiến bước ước lượng số dòng ở mục 3.4.3 phân loại nhầm.

4.3. Kết quả nhận dạng ký tự

4.3.1. Mức ký tự và đóng góp của khối hậu xử lý

Bảng 4.4. Độ chính xác trước và sau hậu xử lý, trên 2.801 biển có nhãn chuỗi

Chỉ số	Ngưỡng tối thiểu	Mục tiêu	Trước	Sau	Chênh
C — đúng mức ký tự	0,92	0,95	0,9061	0,9483 (Cận ngưỡng)	+4,22
CER	$\leq 0,08$	$\leq 0,05$	0,0939	0,0517	—
S₀ → S₁ — đúng cả chuỗi	0,80 → 0,85	0,85 → 0,90	0,6373 (Chưa đạt)	0,7701 (Chưa đạt)	+13,28
Số biển sửa đúng / bị làm hỏng	—	—	—	372 / 0	—
Phân rã lỗi ký tự <i>S / D / I</i> trên <i>N</i> = 23.855	—	—	862 / 1.272 / 107	—	—

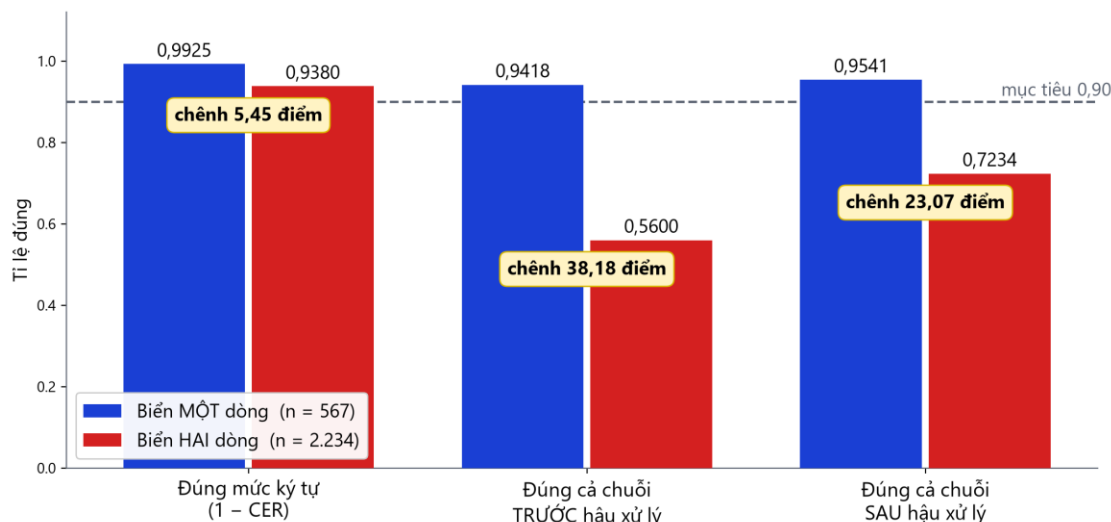
Khối hậu xử lý đóng góp +13,28 điểm, sửa đúng 372 biển và không làm hỏng biển nào (Bảng 4.4). Con số "0 biển bị làm hỏng" không phải may mắn mà là hệ quả của nguyên tắc thiết kế ở mục 3.6d: biểu thức chính quy được thử **trước** khi sửa bất cứ thứ gì, nên chuỗi vốn đã hợp lệ không bao giờ bị can thiệp.

Phân rã lỗi ký tự cho một manh mối quan trọng: **số ký tự bị xoá ($D = 1.272$) lớn hơn số bị thay thế ($S = 862$)**. Hồ sơ lỗi thiên về xoá có cách giải thích tự nhiên là **mất hẳn một dòng** — đúng cơ chế đã dự đoán ở mục 2.4.2 (Bảng 4.5).

4.3.2. Tách theo bố cục — kết quả quan trọng nhất của chương

Bảng 4.5. Độ chính xác nhận dạng tách theo biển một dòng và hai dòng

Chỉ số	Biển một dòng	Biển hai dòng	Chênh (điểm %)
Số mẫu (<i>tổng 2.801</i>)	567	2.234	—
C — đúng mức ký tự	0,9925	0,9380	5,45
S₀ — đúng cả chuỗi, trước hậu xử lý	0,9418	0,5600	38,18
S₁ — đúng cả chuỗi, sau hậu xử lý	0,9541	0,7234	23,07
Cải thiện nhờ hậu xử lý	+1,23	+16,34	—



Hình 4.1. Đối chiếu biển một dòng và hai dòng trên ba chỉ số

Hình 4.1 cho thấy quan hệ phi tuyến ở mục 2.4.3 bằng số thật: cột đo mức ký tự gần như không phân biệt hai bố cục (0,9925 so với 0,9380), nhưng sai một ký tự trong tám là hỏng cả chuỗi, nên chênh lệch 5,45 điểm ở mức ký tự **khuyến đại thành 23,07 điểm** ở mức chuỗi. Ba kết luận rút ra:

1. **Điểm nguyền nằm ở biển hai dòng.** Biển một dòng đạt $S_1 = 0,9541$, vượt cả mục tiêu 0,90; biển hai dòng chiếm 79,8% tập đánh giá nên kéo con số tổng xuống.
2. **Hậu xử lý hiệu quả hơn hẳn trên biển hai dòng** — +16,34 so với +1,23 điểm; biển một dòng vốn đã đọc gần đúng nên còn ít chỗ để sửa.
3. **Tách-ghép đã thu hẹp đáng kể khoảng cách nhưng chưa xoá được** — từ 36,79 xuống 23,07 điểm; phần còn lại nằm ở năng lực mô hình ký tự, vì khâu cắt-ghép đã được đo tách bạch ở mục 4.4.

4.3.3. Ma trận nhầm lẫn ký tự và mức chính xác của bảng luật

Mục 3.6c đã nêu một giới hạn: bảng ánh xạ nhầm lẫn ban đầu **suy từ hình dạng ký tự chứ không từ đo đạc**. Mục này kiểm chứng nó bằng ma trận nhầm lẫn 36×36 đo được, và kết quả đã được dùng để **sửa lại chính bảng đó** (Bảng 4.6).

Bảng 4.6. Mười cặp ký tự bị nhầm nhiều nhất, đối chiếu bảng luật hiện hành

Hạng	Nhầm	Số lần	Tỉ lệ trong tổng lỗi thay thế	Bảng luật có phủ?
1	L → 1	90	10,44%	có — đúng chiều
2	E → F	73	8,47%	không
3	4 → L	53	6,15%	không
4	U → 1	38	4,41%	không
5	D → 0	34	3,94%	có — đúng chiều
6	Z → 7	32	3,71%	không
7	2 → 7	26	3,02%	không
8	X → Y	21	2,44%	không

Hạng	Nhầm	Số lần	Tỉ lệ trong tổng lỗi thay thế	Bảng luật có phủ?
9	B → R	20	2,32%	không
10	9 → 0	19	2,20%	không

Kết quả này là một **phát hiện âm có giá trị**: bảng luật suy từ hình dạng chỉ phủ **2 trong 10** cặp nhầm phổ biến nhất, tuy cả hai đều đúng chiều. Tám cặp không được phủ, trong đó bốn cặp E → F, 4 → L, U → 1, Z → 7 là những cặp mà trực giác hình dạng không gợi ra, nhưng thực tế lại rất phổ biến trên ảnh phân giải thấp.

Phép thay thế **đã được thực hiện ngay trong vòng này** cho hai cặp có đủ bằng chứng: mục L → 1 tại vị trí chữ số đổi thành L → 4 (hạng 3, 53 lần) và mục cho 7 tại vị trí chữ cái đổi thành 7 → Z (hạng 6, 32 lần), nhận dạng chính xác thêm **53 biến đọc đúng và 0 biến hỏng** — đây chính là cấu hình triển khai sinh ra $S_1 = 0,7701$, và bảng ánh xạ phủ **4 trong 10** cặp. Sáu cặp còn lại không sửa được bằng cơ chế này: năm cặp cùng kiểu (E → F, X → Y, B → R, 2 → 7, 9 → 0) không bao giờ kích hoạt mặt nạ vị trí vì ký tự đọc ra vẫn đúng kiểu; cặp U → 1 xung đột với L → 1 phổ biến hơn (90 so với 38 lần) nên giữ chiều thẳng. Xử lý chúng cần một cơ chế khác và ngữ liệu lớn hơn (hướng phát triển số 2, mục 5.3). Đây là ví dụ điển hình cho việc đo đặc thay thế trực giác.

Hiện tượng chữ số 4 bị nhận diện thành chữ cái L chiếm tần suất cao (53 ca, đứng hạng 3 trong tổng số lỗi thay thế, Bảng 4.6) là một phát hiện thị giác đặc thù của bài toán ALPR trên ảnh độ phân giải thấp. Theo chuẩn font chữ đậm nổi của biển số Việt Nam, nét gạch ngang của số 4 có độ dày nhỏ hơn nét sổ dọc. Dưới điều kiện chụp xa, bám bụi hoặc bị ánh sáng tán xạ làm giảm tương phản cục bộ, nét gạch ngang này dễ bị mờ nhòe hoặc đứt đoạn sau bước nhị phân hóa nội bộ của OCR. Khi đó, cấu trúc còn lại chỉ gồm một nét sổ đứng và một đoạn chân ngang bên phải, khiến mô hình trích xuất đặc trưng hiểu nhầm thành chữ L viết hoa.

4.3.4. So sánh ba bộ nhận dạng trên cùng một tầng bao quanh

Câu hỏi: chọn PaddleOCR có đúng không, khi một số tài liệu công khai lại nghiêng về EasyOCR?

Thiết kế thí nghiệm. Cả ba bộ nhận dạng chạy trong **đúng một tầng bao quanh** — toàn bộ chuỗi bước xử lý ảnh bọc quanh bộ nhận dạng, đúng như cấu hình triển khai — trên **cùng một mảng 2.801 vùng biển đã chuẩn bị xong**; khác biệt duy nhất còn lại là bộ nhận dạng. So sánh ba bộ nhận dạng với ba tầng bao quanh khác nhau là đo tầng bao quanh chứ không đo bộ nhận dạng (Bảng 4.7).

Bảng 4.7. So sánh ba bộ nhận dạng trên 2.801 biển số Việt Nam

Bộ nhận dạng	Tắt bước tách-ghép	Có tách-ghép	+ hậu xử lý	Riêng biển 2 dòng
PaddleOCR	28,81%	63,73%	68,87%	62,3%
EasyOCR	6,53%	10,35%	14,28%	10,7%
Tesseract	9,57%	9,60%	10,28%	0,1%

PaddleOCR cao hơn hẳn trong phép đo này — 68,87%, hơn EasyOCR 54,59 điểm và hơn Tesseract 58,59 điểm; khoảng cách quá lớn để quy cho nhiều, nhưng kết luận chỉ áp cho cấu hình đánh giá nêu ở đoạn *Thiết kế thí nghiệm* trên.

Tesseract không đọc được biển hai dòng: 0,1% trên 2.234 mẫu, kể cả sau khi đã ghép thành một dòng, trong khi đọc được 50,4% biển một dòng. Đã kiểm bằng mắt để loại khả năng lỗi công cụ — nó **có** đọc ra chữ nhưng luôn xuất hiện ký tự nhiễu / ký tự giả mạo, và 700/2.801 lần trả chuỗi rỗng.

Kết quả ngoài dự đoán nhất — bước tách-ghép KHÔNG độc lập bộ nhận dạng:

Bộ nhận dạng	Mức tăng nhờ tách-ghép
PaddleOCR	+34,92 điểm
EasyOCR	+3,82 điểm
Tesseract	+0,03 điểm

Nếu cả ba cùng tăng mạnh, đóng góp kỹ thuật ở mục 3.4.5 sẽ là một kỹ thuật độc lập bộ nhận dạng — một khẳng định mạnh hơn nhiều. **Dữ liệu không cho phép nói thế**. Phát biểu đúng là: tách rồi ghép ngang là **điều kiện cần** để đọc biển hai dòng — nó biến bài toán đa dòng thành bài toán một dòng — nhưng **không đủ**; bộ nhận dạng vẫn phải đủ mạnh để tận dụng dải ảnh đã ghép.

Ngược lại, **bộ luật hậu xử lý giúp cả ba** (+5,14 · +3,93 · +0,68 điểm), nên riêng nó **là** một đóng góp độc lập bộ nhận dạng.

4.4. Bóc tách đóng góp của từng bước xử lý ảnh

Mục này là lý do các bước ở mục 3.4 được thiết kế bật tắt độc lập.

4.4.1. Ghép rồi đọc một lần, so với đọc riêng từng nửa

Thí nghiệm A/B trên **200 biển hai dòng** với hạt giống ngẫu nhiên cố định cho kết quả ở Bảng 4.8.

Bảng 4.8. Hai chiến lược đọc biển hai dòng

Phương án	Đúng	Chuỗi rỗng	Thời gian
A — ghép ngang rồi đọc một lần (<i>đang dùng</i>)	129/200 = 64,50%	2	340,11 ms
B — đọc riêng từng nửa rồi nối chuỗi	7/200 = 3,50%	9	391,35 ms

B kém A 61,00 điểm phần trăm và còn tốn thêm 51,24 ms; trong 200 ca, 122 ca A thắng B và **0 ca B thắng A**. Kết quả bác bỏ dứt khoát giả thuyết "đọc riêng từng dòng thì chính xác hơn" và cho thấy chiến lược ghép ngang là **cần thiết** đối với biển hai dòng — cơ chế gây lỗi của phương án B đã phân tích ở mục 3.4.7.

4.4.2. Bậc thang thử lại: Đánh đổi chi phí tính toán khi nhận dạng thêm 34 biển

Bậc thang nắn hình và giãn dọc ở mục 3.4.6 cải thiện thêm **34 biển đọc đúng**. Cái giá đo được:

Bảng 4.9. Ảnh hưởng của bậc thang thử lại lên độ trễ

Chỉ số	Tắt bậc thang	Bật bậc thang (<i>cấu hình triển khai</i>)	Chênh
p50	414,67 ms	405,77 ms	-8,90
p95	866,3 ms	1.143,10 ms	+276,80
p99	—	1.420,07 ms	—

Điểm đáng chú ý: **trung vị thậm chí giảm nhẹ**. Vì bậc thang chỉ chạy **sau khi lần đọc đầu thất bại**, nó không chạm vào trường hợp thường; toàn bộ chi phí dồn vào **đuôi phân bố**. Ở cấu hình đo của thí nghiệm này — **trước** đợt tối ưu tăng suy luận ở mục 4.5.2 — 95% ảnh xong dưới 1,15 giây và một nửa xong dưới 0,41 giây; p50 mô tả trải nghiệm điển hình còn p95 mô tả trường hợp xấu.

Đây là một **thoái lui có chủ ý và đã định lượng**: đổi 277 ms ở p95 lấy 34 biển đọc thêm.

4.4.3. Siêu phân giải: Đánh giá thực nghiệm bậc FSRCNN

Bậc thứ ba của thang thử lại là **siêu phân giải** bằng mạng FSRCNN [15], dành cho vùng biển quá nhỏ. Kết quả đo ở **lượt bóc tách sớm** — một lượt đo riêng, trước lượt đo ở Bảng 4.9 và trên nền máy khác, nên số tuyệt đối không cộng trừ chéo được với Bảng 4.9:

	Chi phí (lượt bóc tách sớm)	Lợi ích
Nắn hình + giãn dọc	+244 ms p95	+34 biển
Siêu phân giải	+319 ms p95, +1.381 ms p99	0 biển

Ở lượt đó, bật cả ba bậc đẩy p95 lên **1.514,26 ms**, tức **vượt cả ngưỡng tối thiểu 1.500 ms**; bậc siêu phân giải vì vậy bị **tắt mặc định** và không có mặt trong cấu hình triển khai — Bảng 4.9 và mục 4.5.2 đều đo cấu hình đã tắt bậc này. Chi phí của nắn hình + giãn dọc đo lại ở lượt Bảng 4.9 là +276,80 ms, hai lượt cho cùng kết luận.

Nhưng số 0 đó phải đọc cho đúng, và đây là điểm phương pháp luận đáng nêu. Cổng vào bậc siêu phân giải chỉ mở cho vùng cắt **nhỏ hơn 200 điểm ảnh**, và trong ngữ liệu đo **0 trên 120 mẫu lọt qua cổng đó**. Cần làm rõ sự khác biệt giữa hai con số thống kê: mẫu số 120 ảnh được đề cập ở đây là tập con các mẫu biển số được đưa vào kịch bản kiểm thử hiệu năng chuyên sâu của đường ống OCR; trong khi con số 262 đối tượng thuộc dải 'Rất nhỏ' tại Bảng 4.3 được thống kê trên toàn bộ tập kiểm thử tổng thể của mô hình phát hiện YOLO11n (nơi hộp bao chiếm dưới 0,5% diện tích khung ảnh 640x640). Trên tập con 120 mẫu thực nghiệm OCR, không có ảnh nào có chiều cao vùng cắt dưới ngưỡng 200 điểm ảnh, dẫn đến việc cổng kích hoạt FSRCNN không được mở trong lượt đo này. Nói cách khác, quyết định tắt dựa trên **"chi phí đã đo, lợi ích chưa ai đo được"** — không phải trên "đã đo

và thấy vô dụng". Mã và công tắc vì vậy được **giữ nguyên**, để đo lại khi có ngữ liệu chứa biến thật sự nhỏ.

Phân biệt này quan trọng: một số 0 do *thiếu điều kiện quan sát* khác hẳn một số 0 do *đã quan sát và thấy bằng không*.

4.5. Hiệu năng

4.5.1. Phân rã ngân sách độ trễ

Bảng 4.10. Phân rã thời gian xử lý một biến số

Bước	Đo được (ms)	% tổng
Giải mã ảnh và tiền xử lý	1,78	1,2%
Suy luận YOLO11n @ 640px	55,66	38,0%
Cắt và tiền xử lý vùng biển	~0,00	0,0%
PaddleOCR (mỗi biển)	89,16	60,8%
Hậu xử lý và kiểm tra hợp lệ	0,03	0,0%
Tổng suy luận thuần	146,63	100%

Ba nhận xét từ Bảng 4.10. **Một, điểm nghẽn là khối nhận dạng ký tự** (60,8%) chứ không phải bộ phát hiện (38,0%). Nguyên nhân: PaddleOCR là một **đường ống nhiều giai đoạn** — phát hiện văn bản, phân loại hướng, rồi mới nhận dạng — thiết kế cho ảnh tài liệu tổng quát, nên hệ thống trả chi phí cho những năng lực mà một vùng biển đã cắt sẵn không cần.

Hai, toàn bộ khối xử lý ảnh của đồ án gần như miễn phí: bước cắt và tiền xử lý vùng biển đo được xấp xỉ 0 ms, hậu xử lý 0,03 ms. Đóng góp +13,28 điểm ở mục 4.3.1 vì vậy đến với chi phí tính toán không đáng kể — một tỉ lệ lợi ích trên chi phí rất hiếm.

Ba, chiến lược tối ưu suy ra trực tiếp từ bảng này. Theo định luật Amdahl, tăng tốc bộ phát hiện gấp 2–3 lần chỉ kéo tổng xuống khoảng 15–23%; muốn giảm mạnh hơn thì khối nhận dạng (60,8%) mới là mục tiêu.

4.5.2. Độ trễ đầu cuối và các chỉ tiêu tài nguyên

Độ trễ một ảnh ở cấu hình triển khai: **p50 = 150,07 ms · p95 = 509,76 ms · p99 = 1.124,13 ms** — đo sau đợt tối ưu tăng suy luận (bật `torch.inference_mode()`, ghim số luồng cho torch và OpenCV, truyền `cpu_threads` xuống bộ nhận dạng). Bảng 4.9 ở trên đo **trước** đợt ấy, nên hai bộ số không được ghép chung: bảng ấy trả lời riêng câu hỏi bậc thang thử lại đắt bao nhiêu. Chỉ tiêu p95 phát biểu ở mức ≤ 1.500 ms (tối thiểu) và ≤ 800 ms (mục tiêu), nên kết luận chính thức là **đạt cả mục tiêu**, với biên 290 ms. Con số này thay cho lượt đo trước tối ưu ở Bảng 4.9 (p95 = 1.143,10 ms), khi chỉ tiêu mới chỉ đạt ngưỡng tối thiểu.

Mọi chỉ tiêu **ngoài đường xử lý ảnh** đều đạt với biên rộng: nạp mô hình 6,41 s (ngưỡng 30 s); bộ nhớ thường trú 0,806 GB (ngưỡng 4 GB); truy vấn 10.000 bản ghi lịch sử 18,71 ms; chạy liên tục 15 phút với **100% thành công trên 5.337 yêu cầu, 0 lỗi — không rò rỉ**.

4.5.3. Độ chính xác bộ phân loại màu nền

Bảng 4.11. Độ chính xác phân loại màu nền trên tập ngoài dữ liệu hiệu chỉnh

Lớp nhãn người gán	Số ảnh	Đúng	Độ chính xác
Biển vàng	694	684	98,56%
Biển trắng	808	787	97,40%
Biển xanh	63	61	96,83%
Tổng	1.565	1.532	97,89%

Bảng 4.11 đo trên 1.565 ảnh ngoài dữ liệu hiệu chỉnh. Ba giới hạn phải nêu kèm. **Một**, 542 ảnh đã bị loại khỏi phép đo — toàn bộ lớp không xác định, cùng các ảnh chụp ban đêm hoặc hồng ngoại mà chính người gán nhãn cũng không xác định được màu. **Hai**, dạng lỗi chủ đạo là **biển trắng bị phân loại thành biển xanh** — 21 trong 33 ca sai — do một số điểm ảnh ám lạnh vượt ngưỡng bão hoà. **Ba**, bộ dữ liệu không chứa biển đỏ và biển ngoại giao nên hai nhánh này chưa có số liệu — ghi thành **hạn chế số 5** ở mục 5.2.

4.6. Phân tích lỗi

Bảng 4.12. Tần suất từng loại lỗi trên 2.801 mẫu, cấu hình triển khai

Mã	Loại lỗi	Số ca	Tỉ lệ trong ca sai	Một dòng	Hai dòng
E1	Nhầm ký tự (<i>thay thế</i>)	392	60,87%	17	375
E2	Thiếu ký tự	76	11,80%	0	76
E3	Thừa ký tự	20	3,11%	5	15
E4	Sai thứ tự	0	0,00%	0	0
E5	Trả chuỗi rỗng	10	1,55%	0	10
E6	Hỗn hợp nhiều loại	146	22,67%	4	142
	Tổng ca sai	644	100%	26	618

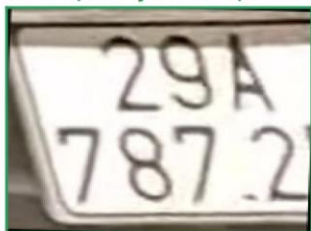
Không có lớp *bỏ sót biển* hay *phát hiện nhầm* vì cả 2.801 mẫu là **vùng biển đã cắt sẵn**, nên bước phát hiện không chạy; hai loại lỗi ấy được đo riêng ở tầng bộ phát hiện.

Bảng 4.12 khép lại mạch lập luận của chương. **Nhầm ký tự chiếm gần hai phần ba số ca sai, và 375 trên 392 ca thuộc biển hai dòng** — cùng một kết luận đã rút ra ở mục 4.3.2, nay xác nhận từ một góc đo khác.

Sai thứ tự bằng 0 là bằng chứng trực tiếp cho thấy thiết kế ghép ngang ở mục 3.4.5 hoạt động đúng: nếu phép ghép đặt nhầm thứ tự hai nửa, hoặc nếu CTC vẫn đọc lộn xộn giữa hai dòng, loại lỗi này phải xuất hiện. Nó không xuất hiện một lần nào.

Thiếu ký tự tập trung tuyệt đối ở biển hai dòng (76/76), khớp với hồ sơ lỗi thiên về xoá ở mục 4.3.1 và với chế độ hồng "mất hẳn dòng trên" mà mục 3.4.7 xử lý.

Ba ca hậu xử lý SỬA ĐƯỢC



nhãn 29A7872
OCR thô 2947872
sau hậu xử lý 29A7872

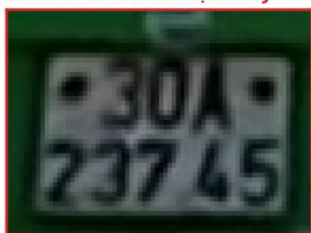


nhãn 78N25203
OCR thô 5203
sau hậu xử lý 78N25203



nhãn 52L26661
OCR thô 52126661
sau hậu xử lý 52L26661

Ba ca VẪN SAI sau hậu xử lý



nhãn 30A3745
đọc ra 37L51U4



nhãn 31F0787
đọc ra 0787



nhãn 52Z8327
đọc ra 52T81327

Hình 4.2. Sáu vùng biển thật: ba ca khối hậu xử lý sửa được, ba ca vẫn sai

Hình 4.2 đặt các con số của Bảng 4.12 lên ảnh thật. Hàng trên là hai cơ chế sửa: mặt nạ vị trí của mục 3.6b (2947872 → 29A7872, 52126661 → 52L26661) và bước phục hồi dòng trên của mục 3.4.7 (5203 → 78N25203). Hàng dưới là ba ca vẫn sai — cả ba đều là biển hai dòng và đều **hỏng ở dòng trên** (30A → 37L, 31F mất hẳn, 52Z → 52T): dòng trên trộn chữ và số ở đúng vị trí mà mặt nạ cho phép cả hai, nên hậu xử lý không có ràng buộc nào để bám vào. Đây là lý do hướng phát triển số 1 ở mục 5.3 nhắm vào bộ nhận dạng chữ không nhắm vào bộ luật.

Cần lưu ý về ảnh: ngữ liệu nhãn xuất mọi vùng cắt về khung vuông 640 × 640, **phá tỉ lệ khung hình gốc**. Hình trên đã khôi phục tỉ lệ bằng đúng hàm mà công cụ đo dùng trước khi chạy nhận dạng. Bước khôi phục này không phải chi tiết trình bày: bỏ nó đi thì S_1 rơi từ 0,7701 xuống **0,4988**, vì mọi vùng cắt vuông đều bị phân loại thành hai dòng.

4.7. Các yếu tố ảnh hưởng tới tính hợp lệ của kết quả

Nguyên tắc: nêu mối đe dọa, đánh giá mức nghiêm trọng, và nói rõ đã làm gì để giảm thiểu — **kể cả khi biện pháp là "không có"** (Bảng 4.13).

Bảng 4.13. Sáu yếu tố ảnh hưởng tới tính hợp lệ

#	Yếu tố	Mức	Biện pháp đã áp dụng
1	Rò rỉ dữ liệu tồn dư không khử được bằng bấm tri giác (mục 3.2.3)	Cao	Nâng ngưỡng gộp 5 → 10 và đo ở nhiều ngưỡng cao hơn. Vẫn còn 791 cặp ở ngưỡng 12; rò rỉ <i>ngữ nghĩa không ngưỡng nào phát hiện được</i> ⇒ mọi chỉ số ở mục 4.2 phải coi là cận trên lạc quan

#	Yếu tố	Mức	Biện pháp đã áp dụng
2	Tập kiểm thử không xuyên bộ dữ liệu	Cao	Không có. Cách đúng là giữ lại một nguồn hoàn toàn không dùng để huấn luyện; chưa thực hiện
3	Mẫu số nhỏ cho chỉ số nhận dạng — 2.801 trên 15.133 ảnh có nhãn chuỗi	Cao	Công bố mẫu số ở mọi bảng; không rút kết luận từ chênh lệch nhỏ
4	Bộ dữ liệu lệch nặng về biển trắng — 97,68% mẫu	Cao	Nêu rõ: kết luận về độ chính xác nhận dạng chỉ áp cho biển trắng
5	Đo trên một cấu hình phần cứng duy nhất	Trung bình	Công bố cấu hình đầy đủ ở mục 4.1; không ngoại suy sang CPU hay hệ điều hành khác
6	Một lượt huấn luyện duy nhất , không ước lượng được phương sai	Trung bình	Cố định hạt giống để tái lập; không phát biểu so sánh dựa trên chênh lệch nhỏ

CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. Kết quả đạt được

Nhóm thực hiện đã xây dựng một hệ thống nhận dạng biển số xe Việt Nam chạy đầu cuối trên máy **không có GPU**, gồm bộ phát hiện tự huấn luyện, khối xử lý ảnh vùng biển, khối nhận dạng ký tự và bộ luật hậu xử lý theo quy chuẩn Việt Nam, kèm một ứng dụng web để trình diễn (Bảng 5.1).

Bảng 5.1. Đối chiếu chỉ tiêu đặt ra với kết quả đo được

Chỉ tiêu đánh giá	Ngưỡng tối thiểu	Mục tiêu	Đo được	
mAP@0,5 · mAP@0,5:0,95 của bộ phát hiện	0,85 · 0,55	0,90 · 0,65	0,9829 · 0,7834	Đạt
Precision · Recall	0,88 · 0,85	0,92 · 0,90	0,9837 · 0,9714	Đạt
C — đúng mức ký tự	0,92	0,95	0,9483	Cận ngưỡng
S₀ → S₁ — đúng cả chuỗi	0,80 → 0,85	0,85 → 0,90	0,6373 → 0,7701	Chưa đạt
Độ trễ một ảnh, p95, trên CPU	≤ 1.500 ms	≤ 800 ms	509,76 ms	Đạt

Hệ thống đạt toàn bộ chỉ tiêu ở tầng phát hiện với biên rộng nhưng chưa đạt chỉ tiêu nhận dạng chuỗi, và phần thiếu hụt **nằm gần như trọn ở biển hai dòng** — biển một dòng đạt S₁ = 0,9541, vượt cả mục tiêu. Các bước xử lý ảnh và hậu xử lý vẫn mang lại cải thiện đáng kể với chi phí tính toán thấp: khối hậu xử lý **+13,28 điểm** (372 biển sửa đúng, 0 biển hỏng, 0,03 ms mỗi biển), bước tách-ghép **+34,92 điểm** cho PaddleOCR — điều kiện cần nhưng không đủ, vì chỉ được +0,03 điểm trên Tesseract (mục 4.3.4). Mọi bước bật tắt được độc lập nên đóng góp của từng bước đo được riêng, và các kết quả âm được ghi lại thay vì bỏ đi.

5.2. Hạn chế

Bảng 5.2 tổng hợp năm hạn chế, xếp theo mức nghiêm trọng.

Bảng 5.2. Năm hạn chế của đồ án

#	Hạn chế	Mức	Hệ quả
1	Nhận dạng biển hai dòng còn yếu	Cao	S ₁ = 0,7234 so với 0,9541 của biển một dòng — điểm nghẽn lớn nhất
2	Bộ dữ liệu lệch nặng về biển trắng (97,68%)	Cao	Kết luận về độ chính xác nhận dạng chỉ áp cho biển trắng
3	Rò rỉ dữ liệu tồn dư không khử được bằng bấm tri giác	Cao	Bấm tri giác tóm tắt bố cục khung ảnh, không tóm tắt chiếc xe (mục 3.2.3)
4	Tập kiểm thử không xuyên bộ dữ liệu	Trung bình	mAP 0,9829 lạc quan hơn mức gặp khi triển khai với nguồn ảnh mới

#	Hạn chế	Mức	Hệ quả
5	Biển đỏ quân đội và biển ngoại giao không có mẫu đánh giá	Trung bình	Bộ dữ liệu không chứa hai loại này, nên hai nhánh phân loại tuy đã cài đặt và chạy đúng trên ảnh demo vẫn chưa có số liệu định lượng

5.3. Hướng phát triển

Sáu hướng phát triển, xếp theo mức tác động, tổng hợp ở Bảng 5.3.

Bảng 5.3. Sáu hướng phát triển, xếp theo mức tác động

#	Hướng	Giải hạn chế	Ghi chú
1	Huấn luyện lại bộ nhận dạng ký tự riêng cho biển số Việt Nam	1	Hướng quan trọng nhất. Phân tích ở mục 4.3.2 đã định vị điểm nghẽn nằm ở năng lực mô hình ký tự, không ở khâu xử lý ảnh
2	Mở rộng bảng ánh xạ nhằm lẫn khi ngữ liệu lớn hơn	1	Vòng đầu đã thực nghiệm và nhận dạng chính xác thêm 53 biển, nâng độ phủ lên 4/10; sáu cặp còn lại (mục 4.3.3) nằm ngoài cơ chế mặt nạ vị trí hoặc xung đột với cặp phổ biến hơn — cần cơ chế sửa theo ngữ cảnh và ngữ liệu lớn hơn
3	Thu thập dữ liệu biển vàng, xanh, đỏ và ngoại giao	2, 5	Điều kiện để mở rộng kết luận ra ngoài biển trắng, và để hai nhánh biển đỏ · ngoại giao có số liệu đánh giá
4	Khử rò rỉ theo chuỗi biển số thay vì theo băm tri giác	3, 4	Gom nhóm theo chuỗi ký tự thay vì theo tương đồng ảnh; giải đúng loại rò rỉ mà pHash không thấy
5	Đo lại bậc siêu phân giải trên ngữ liệu có biển thật sự nhỏ	—	Mục 4.4.3: số 0 hiện tại do thiếu điều kiện quan sát , không phải do đã quan sát thấy vô dụng
6	Tăng tốc khối nhận dạng: lượng tử hoá, đóng gói ONNX hoặc OpenVINO	—	Khối nhận dạng chiếm 60,8% ngân sách độ trễ (Bảng 4.10); bộ phát hiện xuất được sang ONNX/OpenVINO nhưng chưa đo trong khuôn khổ đề án này

5.4. Kết luận chung

Đề tài đặt ra một bài toán có ràng buộc rõ: nhận dạng biển số xe Việt Nam, hỗ trợ **cả biển một dòng và biển hai dòng**, suy luận hoàn toàn trên CPU. Hệ thống đáp ứng ràng buộc vận hành và đạt toàn bộ chỉ tiêu ở tầng phát hiện với biên rộng, nhưng chưa đạt chỉ tiêu độ chính xác ở tầng nhận dạng ký tự.

Xét từ góc độ môn học, kết quả đáng chú ý nhất không phải một con số cao mà là **quan hệ giữa phép biến đổi ảnh và giả định của mô hình**. Bài toán biển hai dòng không được giải bằng cách thay một mô hình mạnh hơn, mà bằng cách **biến đổi ảnh đầu vào cho khớp giả định của mô hình sẵn có**: hạ một ảnh hai dòng thành một dải một dòng, và trong lúc đó tăng gấp đôi số điểm ảnh dành cho mỗi hàng ký tự. Phép biến đổi đó đóng góp **34,92 điểm** — nhiều hơn bất kỳ thay đổi nào khác trong đề án.

Đồng thời, chính phép đo đó cũng chỉ ra giới hạn của cách tiếp cận: nó đóng góp **0,03 điểm** cho Tesseract. Xử lý ảnh dợn đường cho mô hình, nhưng không thay được năng lực của mô hình.

TÀI LIỆU THAM KHẢO

- [1] Báo Dân trí, "Việt Nam có 77 triệu xe máy, cứ 1.000 dân có 770 người sở hữu xe máy," Báo Dân trí, 2024. [Trực tuyến]. Địa chỉ: <https://dantri.com.vn/thoi-su/viet-nam-co-77-trieu-xe-may-cu-1000-dan-co-770-nguoi-so-huu-xe-may-20241104141910472.htm> (truy cập ngày 2026-07-19).
- [2] R. Laroca, E. V. Cardoso, D. R. Lucio, V. Estevam, D. Menotti, "On the Cross-Dataset Generalization in License Plate Recognition," trong *International Conference on Computer Vision Theory and Applications (VISAPP)*, 2022. [Trực tuyến]. Địa chỉ: <https://arxiv.org/abs/2201.00267> (truy cập ngày 2026-07-19).
- [3] Bộ Công an, "Thông tư số 79/2024/TT-BCA quy định về cấp, thu hồi chứng nhận đăng ký xe, biển số xe cơ giới, xe máy chuyên dùng," 2024. [Trực tuyến]. Địa chỉ: <https://chinhphu.vn/?pageid=27160&docid=211945&classid=1&orggroupid=4> (truy cập ngày 2026-07-19).
- [4] Bộ Công an, "Thông tư số 51/2025/TT-BCA sửa đổi, bổ sung một số điều của Thông tư số 79/2024/TT-BCA quy định về cấp, thu hồi chứng nhận đăng ký xe, biển số xe cơ giới, xe máy chuyên dùng," ban hành 30/06/2025, hiệu lực 01/07/2025, Công báo số 887+888 ngày 15/07/2025. [Trực tuyến]. Địa chỉ: <https://congbao.chinhphu.vn/van-ban/thong-tu-so-51-2025-tt-bca-45356/57302.htm> (truy cập ngày 2026-09-03).
- [5] Bộ Công an, "Quy chuẩn kỹ thuật quốc gia về biển số xe QCVN 08:2024/BCA," 2024. [Trực tuyến]. Địa chỉ: <https://mps.gov.vn/chinh-sach-phap-luat/bai-viet/quy-chuan-ky-thuat-quoc-gia-ve-bien-so-xe-d1-t1592> (truy cập ngày 2026-07-19).
- [6] Bộ Công an, "Nhận diện màu sắc, seri, ký hiệu biển số xe của cơ quan, tổ chức, cá nhân từ 01/01/2025," Cổng Thông tin điện tử Bộ Công an, 2024. [Trực tuyến]. Địa chỉ: <https://bocongan.gov.vn/chinh-sach-phap-luat/bai-viet/nhan-dien-mau-sac-seri-ky-hieu-bien-so-xe-cua-co-quan-to-chuc-ca-nhan-tu-01012025-d1-t1617> (truy cập ngày 2026-07-19).
- [7] G. Jocher, J. Qiu, "Ultralytics YOLO11," Ultralytics, 2024. [Trực tuyến]. Địa chỉ: <https://docs.ultralytics.com/models/yolo11/> (truy cập ngày 2026-07-19).
- [8] C. Cui, "PP-OCRv5: A Specialized 5M-Parameter Model Rivaling Billion-Parameter Vision-Language Models on OCR Tasks," *arXiv:2603.24373*, 2026. [Trực tuyến]. Địa chỉ: <https://arxiv.org/html/2603.24373v1> (truy cập ngày 2026-07-19).
- [9] Sutikno, A. Sugiharto, R. Kusumaningrum, "Enhanced Automatic License Plate Detection and Recognition using CLAHE and YOLOv11," 2025.
- [10] K. Zuiderveld, "Contrast Limited Adaptive Histogram Equalization," trong *Graphics Gems IV*, P. S. Heckbert, biên tập. San Diego: Academic Press, 1994, tr. 474–485.

- [11] C. Tomasi, R. Manduchi, "Bilateral Filtering for Gray and Color Images," trong *Proceedings of the Sixth International Conference on Computer Vision (ICCV)*, 1998, tr. 839–846. doi: 10.1109/ICCV.1998.710815.
- [12] A. Graves, S. Fernández, F. Gomez, J. Schmidhuber, "Connectionist Temporal Classification: Labelling Unsegmented Sequence Data with Recurrent Neural Networks," trong *Proceedings of the 23rd International Conference on Machine Learning (ICML)*, 2006, tr. 369–376. doi: 10.1145/1143844.1143891.
- [13] B. Shi, X. Bai, C. Yao, "An End-to-End Trainable Neural Network for Image-Based Sequence Recognition and Its Application to Scene Text Recognition," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, q. 39, s. 11, tr. 2298–2304, 2017. doi: 10.1109/TPAMI.2016.2646371.
- [14] C. Zauner, "Implementation and Benchmarking of Perceptual Image Hash Functions," Luận văn thạc sĩ, Upper Austria University of Applied Sciences, Hagenberg, 2010.
- [15] C. Dong, C. C. Loy, X. Tang, "Accelerating the Super-Resolution Convolutional Neural Network," trong *European Conference on Computer Vision (ECCV)*, 2016, tr. 391–407. doi: 10.1007/978-3-319-46475-6_25.
- [16] G. Bradski, "The OpenCV Library," *Dr. Dobb's Journal of Software Tools*, 2000.

PHỤ LỤC

Phụ lục A. Cấu hình và siêu tham số

A.1. Môi trường thực thi

Chỉ cần khi muốn tái lập **số đo hiệu năng**; số đo độ chính xác không phụ thuộc những giá trị này.

Hạng mục	Giá trị
Hệ điều hành	Windows 11 Pro 10.0.26200
CPU · RAM	Intel Core i5-14600K, 14 nhân / 20 luồng · 31,77 GiB
GPU	Không có GPU CUDA
Python	3.13.12
Thư viện chính	ultralytics 8.4.101 · torch 2.13.0+cpu · paddleocr 3.7.0 · paddlepaddle 3.3.1 · opencv-python 4.10.0.84 · numpy 2.4.5

Phiên bản thư viện trích từ **môi trường đang chạy tại thời điểm đo**, không lấy từ tệp khai báo phụ thuộc — tệp khai báo ghi *ràng buộc phiên bản*, không ghi *phiên bản đã cài*. Riêng phiên bản OpenCV đáng ghi vì quy ước góc của minAreaRect từng đổi giữa các phiên bản lớn, và bước nắn hình ở mục 3.4.6 phải xử lý riêng trường hợp ngoại lệ này.

A.2. Siêu tham số huấn luyện bộ phát hiện

Bảng dưới trích từ tệp tham số do thư viện Ultralytics tự sinh sau lượt huấn luyện chính thức — bản ghi **đã thực thi**, không phải cấu hình dự định.

Tham số	Giá trị
model	yolo11n.pt (tiền huấn luyện COCO, 2.590.035 tham số)
imgsz	640
epochs · batch	20 · 8
optimizer	AdamW
lr0 · lrf · cos_lr	0,001 · 0,01 · bật
momentum · weight_decay	0,937 · 0,0005
device	cpu
seed · deterministic	42 · bật
fliplr	0,0 — tắt hoàn toàn (xem mục 3.3)
flipud	0,0
hsv_h · hsv_s · hsv_v	0,015 · 0,7 · 0,4
translate · scale	0,1 · 0,5
mosaic	1,0

Tham số	Giá trị
Thời gian huấn luyện	30,2 phút/epoch · tổng 36.181 s $\approx$ 10,05 giờ

A.3. Tham số của khối xử lý ảnh

Tham số	Giá trị	Mục
Chiều cao phóng đại vùng biển	64 điểm ảnh	3.4.2
CLAHE — hệ số giới hạn · lưới ô	2,0 · 8 × 8	3.4.2
Ngưỡng tỉ lệ khung hình phân loại số dòng	2,5	3.4.3
Điểm kết thúc nửa trên · bắt đầu nửa dưới	5/12 · 1/3 chiều cao	3.4.4
Chiều cao tối thiểu sau ghép ngang	48 điểm ảnh	3.4.5
Bộ phân loại màu — thu biên · ngưỡng chiếm ưu thế	18% mỗi phía · 30%	3.5
Ngưỡng Hamming khử trùng lặp	10	3.2.2

Phụ lục B. Hướng dẫn cài đặt và chạy

B.1. Chạy bằng Docker (khuyến nghị)

Yêu cầu duy nhất là Docker Desktop. Từ thư mục gốc dự án:

```
cp deployment/.env.example .env
docker compose up -d --build
```

Sau khi hai container báo trạng thái khoẻ mạnh:

- Giao diện web: <http://localhost:5173>
- Tài liệu API tự sinh: <http://localhost:5173/docs>

Dừng hệ thống bằng `docker compose down`. **Không thêm cờ -v** trừ khi thực sự muốn xoá dữ liệu, vì cờ đó xoá luôn volume chứa cơ sở dữ liệu lịch sử.

Trọng số mô hình **không nằm trong ảnh Docker** mà được gắn từ ngoài dưới dạng chỉ đọc, nên tệp `models/best.pt` phải có mặt trước khi khởi động.

B.2. Chạy trực tiếp trên máy, không dùng Docker

Cần Python 3.13 và Node.js 18 trở lên. Đề án dùng **một môi trường ảo duy nhất**. Bản đầu từng tách làm ba, vì bộ phụ thuộc của thư viện nhận dạng ký tự hạ cấp NumPy và thay thư viện thị giác máy tính bằng một biến thể lùi một phiên bản lớn so với nhánh huấn luyện. Cài chung thì mỗi lần cài lại một nhánh sẽ âm thầm đổi phiên bản nhánh kia — lỗi không làm sập chương trình mà làm **kết quả đo không tái lập được**.

```
python -m venv backend/.venv
backend/.venv/Scripts/pip install -r backend/requirements.txt
backend/.venv/Scripts/pip install -r backend/requirements-inference.txt
backend/.venv/Scripts/python -m uvicorn backend.main:app --port 8000
```

Giao diện chạy riêng:

```
cd frontend
npm install
npm run dev
```

B.3. Chạy lại các phép đo của Chương 4

```
backend/.venv/Scripts/python ai/evaluation/ocr_accuracy.py
backend/.venv/Scripts/python ai/evaluation/benchmark_system.py
backend/.venv/Scripts/python ai/evaluation/benchmark_engines.py
```

Ba công cụ này đọc cấu hình từ biến môi trường thay vì tự dựng cấu hình riêng, nên các công tắc bật tắt từng bước xử lý ảnh ở mục 3.4.1 có hiệu lực với chúng. Đây là điều kiện để **bóc tách đóng góp của từng bước** ở mục 4.4; một công cụ đo tự dựng cấu hình riêng sẽ đo một hệ thống khác với hệ thống được triển khai.

B.4. Dựng lại quyền báo cáo

```
backend/.venv/Scripts/python scripts/build_thesis.py
powershell -File scripts/export_thesis_pdf.ps1
```

Lệnh đầu ghép năm chương thành một tệp Markdown rồi kết xuất .docx; thứ tự ghép khai báo ở ORDER.txt trong chính thư mục đó. Lệnh hai dùng Word để kết xuất PDF và điền số trang cho mục lục. Cả hai đã đặt sẵn bản đồ án môn học làm mặc định nên không cần tham số; muốn dựng bản khác thì truyền --src cho lệnh đầu và -Nguon cho lệnh hai.

Phụ lục C. Phân công công việc

Thành viên	MSSV	Công việc chính
Phạm Nguyễn Thế Châu	25410004	Thu thập dữ liệu, huấn luyện YOLO11n, đánh giá bộ phát hiện
Nguyễn Công Hậu	25410006	Xây dựng giao diện React, FastAPI, Docker, kiểm thử tích hợp
Nguyễn Minh Hiếu	25410007	Thiết kế đường ống xử lý ảnh, hậu xử lý biến số, thực nghiệm OCR, viết báo cáo
Phạm Công Thành	25410013	Xây dựng bộ dữ liệu, khử trùng lặp, đánh giá và phân tích kết quả

Các hạng mục trên là **phần việc chính** của từng thành viên, không phải ranh giới tuyệt đối: những quyết định ảnh hưởng tới số liệu công bố — chọn ngưỡng phân loại bố cục (mục 3.4.3), dùng lượt tinh chỉnh bộ nhận dạng, chốt cấu hình triển khai — đều do cả nhóm thống nhất.